

RepoLens

An AI-powered Repository Intelligence Platform for rapid comprehension of unfamiliar software systems.

Understand software systems, not just source code.

- Product Requirements Document
- Software Requirements Specification
- Engineering Design Document
- Technical Architecture
- Academic Proposal

VERSION	ARCHITECTURE	STACK	INFRASTRUCTURE	STATUS
2.0 — Expanded	Modular Monolith	FastAPI · Next.js 15	\$0.00 Free Tier	Approved for Build

§ Document Control

This document is the single normative reference for the RepoLens system. It supersedes all prior design notes. Where an implementation detail conflicts with this document, this document is authoritative until formally revised.

Revision History

Version	Stage	Author	Summary of change
0.1	Concept	Engineering team	Initial problem framing, competitive scan, product concept. No architecture committed.
0.4	Design	Engineering team	Two-tier pipeline concept established. Tree-sitter selected over native parsers. Module detection moved from LLM inference to deterministic clustering after early hallucination findings.
1.0	Frozen	Engineering team	Complete architecture, AI pipeline, database schema, REST surface, milestone plan. Scope boundaries fixed.
2.0	Expanded	Engineering team	Current. Restructured as a combined PRD / SRS / EDD / architecture reference. Adds architecture decision records, threat model, observability and failure-recovery strategy, cost model, performance budgets, expanded API and data dictionary, testing matrix, glossary and appendices. No decision from v1.0 was reversed; every addition is explanatory or additive.

Document Conventions

Requirement language follows IETF RFC 2119 conventions. **MUST** denotes an absolute requirement; **SHOULD** denotes a strong recommendation that may be departed from with recorded justification; **MAY** denotes an optional capability. Identifiers of the form `FR-nn` denote functional requirements, `NFR-nn` non-functional requirements, and `ADR-nnn` architecture decision records. All are globally unique within this document and are referenced rather than restated.

INTENDED READERS

Reviewers assessing product merit should read Parts I and II. **Engineers assessing architecture** should read Parts III and IV. **Implementers** should read Parts V through VII, which are written to be sufficient for construction without further clarification. **Automated agents** ingesting this document as a knowledge base should treat Part V (schema, API, algorithms) and the appendices as the machine-consumable core.

SELF-SUFFICIENCY GUARANTEE

This specification is written to be read in isolation. Every algorithm is stated in enough detail to be implemented, every schema is given as executable DDL, every endpoint is given with its request and response contract, and every non-obvious decision carries the alternatives that were rejected. No external document is required to build the system described here.

Contents

PART I — STRATEGIC FOUNDATION

- 1 **Executive Summary** problem, solution, and outcome in one page
- 2 **Product Vision and Convictions** the three beliefs that shape the architecture
- 3 **The Comprehension Problem** why unfamiliar codebases resist understanding
- 4 **Competitive Landscape** why the gap persists and where RepoLens sits
- 5 **Engineering Philosophy** deterministic grounding as an operating constraint
- 6 **Goals, Non-Goals and Scope** what is committed and what is refused
- 7 **Users and Personas** four readers, four distinct entry surfaces

PART II — PRODUCT REQUIREMENTS

- 8 **Use Cases** primary and secondary interaction scenarios
- 9 **Functional Requirements** FR-01 through FR-15 with verification
- 10 **Non-Functional Requirements** NFR-01 through NFR-08 and their derivation
- 11 **User Workflow and Product Design** journey, capabilities, error surfaces
- 12 **Acceptance Criteria and Edge Cases** definition of done, failure handling

PART III — SYSTEM ARCHITECTURE

- 13 **Architecture Overview** modular monolith, layers, responsibilities
- 14 **Architecture Decision Records** eight recorded decisions with alternatives
- 15 **Data Flow and Processing Lifecycle** the pipeline end to end
- 16 **Persistence Strategy** system of record versus derived index
- 17 **Infrastructure and Deployment** three free tiers, one topology
- 18 **Security Architecture and Threat Model** boundaries, controls, coverage
- 19 **Performance Engineering** latency budget and scalability path
- 20 **Observability** emission, collection, interpretation
- 21 **Failure and Disaster Recovery** degradation modes and recovery posture
- 22 **Cost Model** free-tier budget and the levers that protect it

PART IV — AI ENGINEERING

- 23 **Two-Tier Intelligence Pipeline** the determinism boundary

- 24 **Static Analysis with Tree-sitter** extraction targets and known limits
- 25 **Module Detection Algorithm** two deterministic passes, then naming
- 26 **Semantic Chunking** why boundaries decide retrieval quality
- 27 **Embeddings and Vector Indexing** model choice, normalisation, rebuild
- 28 **Hierarchical Map-Reduce Summarisation** three stages, bounded context
- 29 **Grounded RAG and Hallucination Control** the gate that precedes the model
- 30 **Repository Map Layout Engine** graph construction and hierarchical layout
- 31 **Prompt Engineering Standards** structural rules for every model call

PART V — IMPLEMENTATION REFERENCE

- 32 **Technology Stack and Rejected Alternatives** every selection, justified
- 33 **Database Design and Data Dictionary** entities, columns, indices, DDL
- 34 **REST API Specification** ten endpoints with full contracts
- 35 **Authentication and Authorization** OAuth flow, JWT sessions, scopes
- 36 **Background Processing and Concurrency** task model and its limits
- 37 **Caching Strategy** SHA-gated invalidation
- 38 **Report Compilation Engine** one model, two rendering paths
- 39 **Frontend Architecture** routing, state, and the map subtree

PART VI — QUALITY AND EVALUATION

- 40 **Testing Strategy** four layers and the CI gate
- 41 **Evaluation Framework** benchmark suite and groundedness audit
- 42 **Performance Benchmarks** targets by repository size class

PART VII — EXECUTION

- 43 **Team Ownership** two engineers, one shared contract
- 44 **Implementation Roadmap** seven milestones across twelve weeks
- 45 **Risk Register** likelihood, impact, mitigation

PART VIII — FORWARD DIRECTION

- 46 **Future Roadmap** the constraint blocking each next step
- 47 **Versioning and API Evolution** how change is absorbed
- 48 **Multi-Language Expansion Path** the four-step recipe

49 Open Research Questions what this design does not yet answer

APPENDICES

- A Glossary** domain and system terminology
 - B Acronyms** expansion reference
 - C Complete ADR Index** all decisions, consolidated
 - D API Quick Reference** single-page endpoint card
 - E Database DDL** executable schema
 - F References** citations and source specifications
-

PART ONE

01 Strategic Foundation

Why this system exists, what problem it addresses, who it serves, and the convictions from which every later architectural decision follows.

-
- 1 Executive Summary
 - 2 Product Vision and Convictions
 - 3 The Comprehension Problem
 - 4 Competitive Landscape
 - 5 Engineering Philosophy
 - 6 Goals, Non-Goals and Scope
 - 7 Users and Personas

1. Executive Summary

RepoLens converts a public GitHub repository into a structured, interactive engineering knowledge model — a Repository Intelligence artefact that answers the architectural questions a developer must resolve before they can contribute, and answers them in minutes rather than days.

Software repositories have grown faster than the tooling that helps people understand them. A developer joining an unfamiliar codebase spends hours tracing directory hierarchies, following import chains, inferring which framework conventions are in play, and hunting for the entry point that makes the rest legible. This work is repeated by every new hire, every open-source contributor, and every engineer rotating onto an unfamiliar service. It is expensive, it is almost entirely undocumented, and the tools that exist address adjacent problems rather than this one.

The adjacent tools are good at what they do. Code completion systems predict the next token inside the file you already have open. Code search engines resolve exact symbols across millions of lines. Static analysers find defects against a rule catalogue. None of them produce the thing a newcomer actually needs: a system-level description of what the modules are, what each is responsible for, how they depend on one another, and where to start reading.

RepoLens produces exactly that artefact. It clones a repository into an isolated workspace, parses every supported source file with Tree-sitter to extract a factual structural record, clusters files into logical modules using two deterministic passes over directory topology and import cohesion, chunks the code along syntax-tree boundaries, embeds those chunks with a CPU-resident model, indexes them for similarity retrieval, and only then invokes a language model — to name, describe, and explain a structure that has already been established as fact. The resulting model is persisted once and serves six interactive capabilities: a repository overview, an interactive module map, a concept-level semantic explorer, metric-grounded insights, a grounded question-answering interface, and exportable PDF and Markdown reports.



The system's defining constraint is that **artificial intelligence is never permitted to invent structure**. Facts about a repository — which files exist, what imports what, where a class is declared — are extracted deterministically and are reproducible across runs. Generation is confined to explaining those facts. When the retrieval layer cannot find evidence that clears a similarity threshold, the system declines to answer rather than producing a plausible guess. This is not a conservative default bolted on late; it is the organising principle from which most of the architecture in Part III and Part IV follows.

THE ONE-SENTENCE ARCHITECTURE

A deterministic static-analysis tier establishes ground truth; a generative tier explains it; PostgreSQL is the sole system of record; FAISS is a disposable index derived from PostgreSQL; and every generative output is either cited or withheld.

RepoLens is built by two engineers over roughly twelve weeks on free-tier infrastructure with no GPU and no paid API. Those constraints are not incidental — they are load-bearing design inputs. The modular monolith, the

CPU-resident embedding model, the choice to rebuild the vector index from the relational store rather than persist it, and the Map-Reduce summarisation strategy that bounds model context regardless of repository size are each traceable to a specific constraint stated in §6.

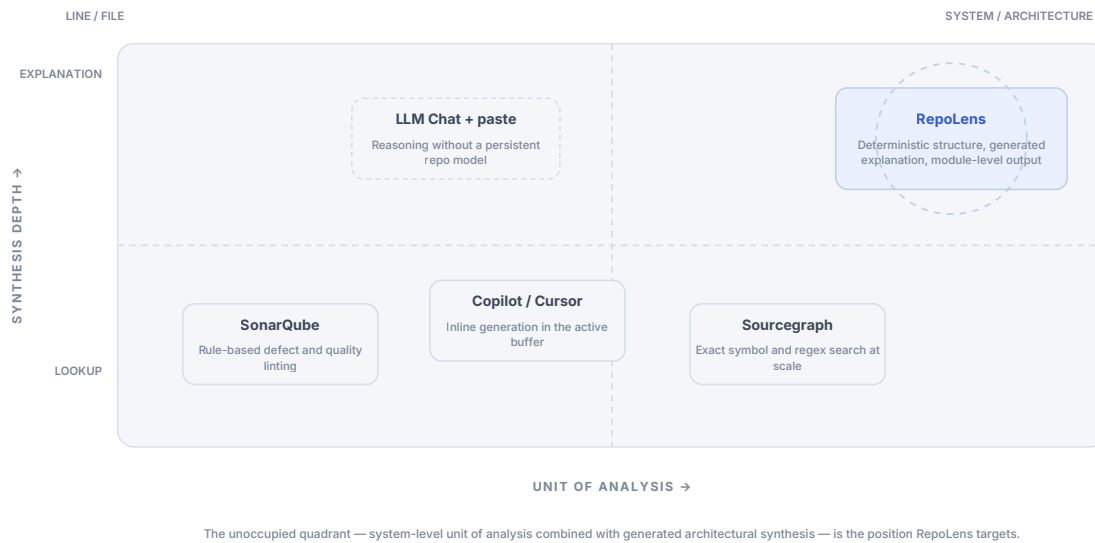


Figure 1.1 — Competitive positioning by unit of analysis and synthesis depth. Existing tools cluster in the lower-left and lower-right: they operate on small units, or they retrieve rather than explain. The upper-right quadrant — system-level unit combined with generated architectural synthesis — is unoccupied, and is the position this system targets.

2. Product Vision and Convictions

The industry has spent a decade making code faster to write. The larger and less glamorous problem — understanding what already exists — has been left to README files that go stale within weeks of being committed.

The vision for RepoLens is narrow on purpose: reproduce, in minutes, the mental model a competent engineer would otherwise assemble after a full day of manual exploration. This is not a claim that a model understands code better than a human does. It is a claim that most of the work involved in orienting inside a new codebase is mechanical, repetitive, and therefore compressible — and that the small remainder which genuinely requires judgement is better served by explanation grounded in extracted fact than by open-ended generation.

Three convictions follow from this, and each has a direct architectural consequence. They are stated here rather than in an architecture chapter because they are premises, not conclusions: the architecture is downstream of them.

2.1 Structure before language

A repository's architecture is a fact, not an opinion. The set of files is enumerable. The import graph is derivable. The location of a class declaration is unambiguous. Facts of this kind should be extracted by a parser, not inferred by a model, because a parser is reproducible and a model is not.

Architectural consequence: the entire Tier 1 pipeline (§23–§27) exists before any model is invoked, and every model call in Tier 2 consumes Tier 1 output as structured input rather than reading raw repository content.

2.2 Modules, not files

No engineer holds a mental model of a codebase at file granularity. They hold it at the level of the auth module, the data layer, the API surface. A tool that presents three thousand files has restated the problem rather than solved it.

Architectural consequence: the module is the primary unit throughout — it is the node type in the repository map, the grouping key for summarisation, the context envelope for retrieval, and a first-class table in the schema. Individual files are deliberately excluded from the graph view (§11.5) to prevent the visualisation from reproducing the very clutter it exists to eliminate.

2.3 Trust is the product

A tool that occasionally invents a plausible but false architectural claim is worse than a tool that says it does not know. False confidence is more expensive than absent information, because a developer will act on the former and investigate around the latter.

Architectural consequence: every generative surface has an engineered path to decline. The grounding gate in §29 aborts before the model is called, not after its output is filtered — a distinction that matters because a prompt instruction can be disregarded by a model whereas a branch in application code cannot.

DEVELOPER NOTE — WHY THIS MATTERS DURING IMPLEMENTATION

These convictions are the tiebreaker when an implementation shortcut presents itself. It will at various points be faster to let the model infer something that Tier 1 could have extracted. The correct response is to extend Tier 1. If a change moves a fact from the deterministic tier into the generative tier, it is a regression regardless of how much code it saves.

3. The Comprehension Problem

Before a developer writes a single meaningful line in an unfamiliar repository, they must implicitly answer five questions. Every one of them is architectural, and none of them are answered by the tools most developers have open.

1. How is this project organised, and what are its logical sub-modules?
2. Where are the entry points, route definitions, and configuration boundaries?
3. Which frameworks, libraries and conventions are actually in use?
4. How do components communicate, and what depends on what?
5. Where should I start reading in order to understand one specific capability?

3.1 The bootstrapping paradox

Existing approaches share a structural failure: they require the developer to already possess a partial mental model in order to use them effectively. A search query for the authentication module presupposes knowing that such a module exists and roughly what it is called. This is the central constraint that shapes the product surface — every primary capability must be usable at zero prior context.

DESIGN CONSEQUENCE

The repository overview and the module map are both zero-input views: they render fully without the user typing anything. Search and question-answering, which require the user to know what to ask, are deliberately positioned as secondary capabilities rather than as the product's entry point. A system that led with a chat box would fail the bootstrapping test.

3.2 Why documentation does not close the gap

Documentation decays because it is maintained by convention rather than enforcement. A README describes the repository at the moment someone last cared to update it; the code describes the repository now. Any comprehension tool that depends on prose written by humans inherits that staleness. RepoLens treats the README as one input among many — useful for stated intent, never trusted for structure — and derives everything structural from the source itself, which cannot be out of date with respect to itself.

3.3 Failure modes of the adjacent tool categories

Category	Representative	Optimised for	Structural blind spot
AI code completion	Copilot, Cursor, Amazon Q	Next-token prediction, localised edits	Context is the active buffer, not the system. No persistent cross-file model survives the session.
Code search	Sourcegraph	Exact symbol, regex, structural queries	Retrieval without synthesis. Presupposes the user knows what to search for.
Static analysis	SonarQube	Defect and vulnerability rules, coverage	Describes what is wrong, never what the system is or why it is shaped that way.
Doc generators	Doxygen, Sphinx	Symbol-level API reference	Documents leaves, not the tree. No module abstraction, no dependency narrative.
General LLM chat	Pasted-code conversations	Open-ended reasoning over supplied text	No repository-scale context, no persistent structure, and no grounding guarantee.

Each blind spot is a consequence of the category's optimisation target rather than an oversight, which is why none of these products is likely to close the gap incidentally — the analysis in §4 develops this point.

4. Competitive Landscape

4.1 Capability comparison

TABLE 4.1 — CAPABILITY MATRIX AGAINST ADJACENT TOOLING

Capability	Copilot / Cursor	Sourcegraph	SonarQube	RepoLens
Primary focus	Code generation and editing	Exact code and symbol search	Quality and security linting	System-level repository understanding
Unit of analysis	Lines, active file	Symbol, regex match	Defect, code smell	Logical module, system architecture
Static parsing	Partial, editor buffer	Full AST indexing	Rule-based analysis	Tree-sitter AST and import extraction
Module map	None	Limited code graph	File tree	Interactive module graph, Dagre + React Flow
Semantic search	Workspace vector index	Keyword and structural	None	bge-small-en-v1.5 with FAISS retrieval
Architectural synthesis	None	None	None	Three-stage Map-Reduce via Gemini 2.5 Flash
Report export	None	None	PDF quality reports	PDF via WeasyPrint, GFM Markdown
Onboarding path	None	None	None	Generated ordered reading plan
Hallucination control	Not applicable	Not applicable	Not applicable	Hard similarity gate at 0.60 cosine

4.2 Why the gap persists

It is worth explaining rather than asserting why this combination remains unserved, because a gap that exists by accident closes quickly and a gap that exists for structural reasons does not.

Completion vendors are economically anchored to the active-file context window. Their success metric — accepted-suggestion rate — is measured per keystroke, and expanding context to repository scale increases latency and cost per suggestion without improving that metric. The incentive points away from system-level understanding.

Search vendors are built on exact-match indexing infrastructure. Adding natural-language architectural synthesis is not a feature addition to that stack; it is a second stack with different storage, different retrieval semantics, and a generative dependency their architecture was not designed to carry.

Static analysis vendors operate rule engines tuned for defect classes. Positive architectural description is not expressible as a rule catalogue — there is no violation to detect when a module is simply well-organised.

HONEST CAVEAT

None of this makes the position defensible in a commercial sense. A sufficiently motivated incumbent could build this. The claim is narrower and sufficient for the purpose: the combination is currently unserved, the problem is real and measurable, and building it requires solving genuine engineering problems rather than assembling existing products. For a system whose objective is engineering demonstration rather than market capture, that is the relevant bar.

5. Engineering Philosophy

Two statements govern the system. Both are deliberately short, because a philosophy that needs a paragraph to state will not survive contact with a scope discussion.

PRODUCT PHILOSOPHY

Developers should spend their time understanding software systems, not searching through source code.

Every interface element, visualisation, and generated output must reduce the cognitive cost of comprehending an unfamiliar codebase. Anything that does not is excluded — this is the operating test applied to every candidate feature in §6.2.

ENGINEERING PHILOSOPHY

RepoLens is an engineering system first. Artificial intelligence is an enabling component, not the product.

AI is applied where summarisation, natural-language reasoning, or semantic similarity genuinely adds value. Where a task is solvable deterministically — extracting imports, enumerating files, locating entry points — static analysis is strictly preferred.

5.1 The deterministic grounding constraint

Stated formally, and treated throughout this document as close to inviolable:

CONSTRAINT DG-1

Static analysis establishes factual structure. Generative models **MUST** be restricted to explaining and summarising that structure. A generative model **MUST NOT** be the origin of any claim about what exists in a repository, what imports what, or where a boundary lies.

5.2 Why this is not negotiable

It would be materially simpler to skip static analysis and ask a model to read a file tree and infer the architecture. This was considered and rejected; ADR-003 records the decision in full. The short reason is that a model operating without a deterministic anchor produces architecturally plausible but factually incorrect module boundaries under exactly the conditions where a comprehension tool is most needed — repositories with unconventional layouts and thin documentation. Plausible-but-wrong is the worst possible output for this product, because it is indistinguishable from correct without doing the manual work the tool was supposed to eliminate.

The cost of the constraint is real: the two-pass clustering algorithm in §25 is meaningfully more engineering than a prompt would have been. The benefit is that module boundaries are traceable to concrete import evidence and are identical across runs, which is what makes the output auditable.

6. Goals, Non-Goals and Scope

6.1 Goals

TABLE 6.1 — COMMITTED GOALS AND THEIR RATIONALE

ID	Goal	Why it is committed
G1	Analyse any supported public repository up to 3,000 files	Defines the operating envelope every pipeline stage is engineered and benchmarked against.
G2	Deterministically extract ASTs, imports, exports, language boundaries	Anchors all downstream generation in verifiable fact per constraint DG-1.
G3	Detect and cluster files into logical modules	Matches the unit engineers actually reason in (conviction 2.2).
G4	Persist one structured Repository Intelligence model	Enables caching, cross-session reuse, and a stable contract between backend and every consumer.
G5	Render an interactive module-level architecture graph	Provides the zero-input comprehension entry point required by §3.1.
G6	Provide concept-based semantic search	Removes the must-already-know-the-name barrier that defeats keyword search.
G7	Answer questions grounded in cited evidence	Makes the intelligence model interactive without surrendering the trust guarantee.
G8	Export PDF and Markdown reports	Makes the model portable beyond the web application, which is where onboarding actually happens.
G9	Generate a recommended reading path	Operationalises the product philosophy into a concrete artefact a newcomer can follow.

6.2 Non-goals

Each of the following was a genuine candidate and each was rejected using the same test: does this increase repository understanding more than it increases engineering risk within a fixed twelve-week, two-engineer, zero-budget constraint?

TABLE 6.2 — REFUSED SCOPE AND THE REASON FOR REFUSAL

ID	Non-goal	Reason for exclusion
NG1	Code generation, completion, inline editing	Different category. Conflating understanding with generation dilutes the product philosophy and roughly doubles the engineering surface.
NG2	Pull request or CI/CD automation	Requires write scopes, which directly contradicts the read-only security model in §18. The security posture is worth more than the feature.
NG3	Private repository support in v1.0	Higher-trust scopes and credential handling disproportionate to the timeline. Deferred, not abandoned — see §46.
NG4	Multi-repository or cross-repository analysis	Multiplies data model and UI complexity for a capability with no established v1.0 demand.
NG5	Multi-agent orchestration frameworks	The Map-Reduce pipeline (§28) is simpler, more debuggable and more cost-predictable for this problem shape. Agent frameworks add orchestration without a capability gain here.
NG6	C/C++, Rust, Go, Ruby parsing in v1.0	Each language needs a grammar integration plus language-specific extraction rules. Four languages done well beats eight done poorly.

6.3 Constraints

TABLE 6.3 — FIXED PROJECT CONSTRAINTS TREATED AS DESIGN INPUTS

Constraint	Value	Design consequence
Team	Two engineers	Modular monolith over microservices (ADR-001); the coordination cost of service boundaries buys nothing at this size.
Timeline	~12 weeks, seven milestones	Scope fixed early and defended; stretch items live in §46 rather than in the build.
Budget	\$0.00, free tiers only	CPU-resident embeddings, no managed vector DB (ADR-002), hosted LLM over self-hosted (ADR-005).
Hardware	No GPU	Embedding model selected for CPU inference quality-per-compute (§27.1).
Languages	Python, Java, JavaScript, TypeScript	Four Tree-sitter grammars; extraction rules per language in §24.1.
Repository size	500–3,000 files	Bounds worst-case parse cost, chunk count, index size and storage footprint (§22.1).
Access model	Public repositories, read-only scopes	Eliminates an entire class of credential risk from the threat model (§18).

7. Users and Personas

Four readers, each entering the product through a different surface. The personas exist to keep feature debates concrete: a proposed capability that serves none of them is out of scope by construction.

7.1 The open-source contributor

Situation: has found an issue worth fixing in a repository they have never opened. **Need:** locate the correct module and file without reading the whole codebase. **Success signal:** time from landing on RepoLens to opening the right file in their own editor is measured in minutes. **Primary surface:** repository map, then semantic explorer.

7.2 The onboarding engineer or intern

Situation: assigned an unfamiliar internal or reference codebase in their first week. **Need:** the shape of the whole system before the first task, plus a defensible order to read things in. **Success signal:** can describe the major modules and their responsibilities after one session. **Primary surface:** overview and learning path, with the exported report as offline reference.

7.3 The reviewing engineer or technical evaluator

Situation: assessing a codebase they did not write — a review, an audit, a candidate's submission. **Need:** a fast, structured, evidence-grounded read on organisation and architectural quality. **Success signal:** can form and defend an opinion after one session. **Primary surface:** insights and export.

7.4 The developer investigating a dependency

Situation: uses a library and needs to know how it actually works internally, beyond its public documentation. **Need:** targeted answers to specific questions without manually tracing call chains. **Success signal:** correctly grounded, cited answer faster than manual reading. **Primary surface:** grounded question answering.

DEVELOPER NOTE — PERSONA COVERAGE TEST

Personas 7.1 and 7.2 both require zero-input surfaces; personas 7.3 and 7.4 require query-driven surfaces. This is the concrete reason the product ships both categories in v1.0 rather than sequencing one after the other — shipping only query-driven capabilities would leave half the user base unable to start.

PART TWO

02

Product Requirements

The complete specification of behaviour: use cases, functional and non-functional requirements, the user workflow, and the acceptance criteria against which the system will be judged complete.

-
- 8 Use Cases
 - 9 Functional Requirements
 - 10 Non-Functional Requirements
 - 11 User Workflow and Product Design
 - 12 Acceptance Criteria and Edge Cases

8. Use Cases

8.1 Primary use cases

UC-1 · Analyse a new public repository

Actor: any authenticated user. **Precondition:** repository is public and within the size envelope. **Trigger:** user selects or pastes a repository URL. **Main flow:** the system resolves the remote HEAD commit SHA, finds no matching prior analysis, enqueues a background job, and streams eight-stage progress to the client until the Repository Intelligence model is persisted. **Postcondition:** a complete intelligence model exists and the dashboard is populated. **Alternate flow:** SHA matches an existing record and the cached model is served without any analysis (see UC-5 inverse).

UC-2 · Explore architecture visually

Actor: user with a completed analysis. **Trigger:** opens the repository map. **Main flow:** the client requests the pre-computed node and edge graph with Dagre-assigned coordinates and renders it; the user pans, zooms, and opens module drawers to inspect file listings and exported symbols. **Postcondition:** user can name the modules and describe their dependency relationships without having opened a source file.

UC-3 · Ask a targeted question

Actor: user with a completed analysis. **Trigger:** submits a natural language question. **Main flow:** query is embedded, top-K chunks retrieved, the similarity gate evaluated; if cleared, a grounded answer with file and line citations is generated. **Alternate flow:** gate not cleared — the standardised insufficient-evidence response is returned and no model call is made.

UC-4 · Export a portable report

Actor: user with a completed analysis. **Trigger:** requests PDF or Markdown export. **Main flow:** the report engine reads the persisted intelligence model, renders it through the appropriate template, and returns a downloadable artefact. **Postcondition:** a self-contained document exists outside the application.

8.2 Secondary use cases

ID	Use case	Description and significance
UC-5	Re-analyse an updated repository	Triggered when the remote SHA has moved since the last analysis. Exercises cache invalidation and confirms that stale intelligence is never served for changed code.
UC-6	Concept search across the codebase	A discovery-oriented variant of UC-3, surfaced through the global command palette. Differs in intent: exploration rather than a specific answer.
UC-7	Follow the generated learning path	A guided variant of UC-2 where reading order is prescribed rather than freely explored. Primary path for persona 7.2.
UC-8	Inspect a single module in depth	Drill-down from the map into file listings, exported symbols and internal dependencies. The bridge between system-level and file-level comprehension.

9. Functional Requirements

Fifteen requirements, each traceable to exactly one goal in §6.1 and implemented by exactly one subsystem in Part IV or Part V. This traceability is deliberate: it lets the test matrix in §40 map suites onto requirements rather than onto implementation details that will shift during construction.

TABLE 9.1 — FUNCTIONAL REQUIREMENTS FR-01 TO FR-15

ID	Requirement	Goal	Priority	Verification
FR-01	The system MUST authenticate users via GitHub OAuth 2.0	G1	Must	JWT issued and validated against a signed session; scope grant inspected
FR-02	The system MUST retrieve the user's accessible public repositories	G1	Must	GitHub REST integration test with a fixture account
FR-03	The system MUST compare commit SHAs and serve cached intelligence when unchanged	G4	Must	Repeat-selection test asserts zero background jobs enqueued
FR-04	The system MUST clone repositories into isolated temporary workspaces	G1	Must	Directory lifecycle asserted on both success and exception paths
FR-05	The system MUST parse Python, Java, JavaScript and TypeScript via Tree-sitter	G2	Must	Node traversal against per-language fixture files
FR-06	The system MUST extract imports, exports, classes, functions and signatures	G2	Must	Extractor unit tests with expected-record assertions
FR-07	The system MUST detect module candidates via two-pass topology and cohesion clustering	G3	Must	Clustering determinism test — identical input yields identical output across runs
FR-08	The system MUST produce AST-boundary-aligned chunks of 300–500 tokens with 50-token overlap	G6	Must	Chunk size distribution and boundary-integrity assertions
FR-09	The system MUST compute 384-dimensional embeddings via bge-small-en-v1.5	G6	Must	Output shape and normalisation verified on CPU inference
FR-10	The system MUST persist vectors in PostgreSQL and reconstruct the FAISS index on demand	G4	Must	Index rebuilt from a cold process; retrieval parity asserted
FR-11	The system MUST execute three-stage Map-Reduce summarisation	G3	Must	Schema validation on every stage response; token accounting logged
FR-12	The system MUST compute 2D module graph coordinates via Dagre	G5	Must	Graph output schema validated; no overlapping node assertion
FR-13	The system MUST execute semantic search over the FAISS index	G6	Must	Retrieval integration test against a seeded index
FR-14	The system MUST answer questions grounded in retrieved evidence and MUST abort generation below 0.60 cosine similarity	G7	Must	Gate unit test asserts zero model invocations below threshold
FR-15	The system MUST export PDF and GitHub-Flavored Markdown reports	G8	Must	Artefact generation test; content parity between both formats asserted

ON FR-14'S WORDING

Note the phrasing: MUST abort generation, not MUST suppress output. The distinction is normative. An implementation that calls the model and discards the answer satisfies the weaker reading and fails this requirement — it incurs the cost and retains the risk that a partially-streamed response leaks to the client.

10. Non-Functional Requirements

TABLE 10.1 — NON-FUNCTIONAL REQUIREMENTS NFR-01 TO NFR-08

ID	Category	Target	Verification method
NFR-01	Processing latency	Repositories under 1,500 files complete analysis in under 180 seconds	End-to-end background task timing across the ten-repository benchmark suite
NFR-02	Search latency	Semantic search returns in under 800 ms	Vector retrieval timing with a warm index
NFR-03	Q&A latency	Grounded answers returned in under 4.0 seconds	Gateway response timing, excluding client render
NFR-04	Storage reliability	100% of temporary workspaces deleted on exit, including on exception paths	Context manager lifecycle assertions plus filesystem audit after fault injection
NFR-05	Access security	Read-only GitHub scopes at all times; no write capability at any layer	OAuth grant inspection; negative test asserting write endpoints are unreachable
NFR-06	Usability	Fully responsive at viewports $\geq$ 1024 px	Cross-browser layout verification at three breakpoints
NFR-07	Maintainability	Strict package separation between the three backend engines; no circular imports	Static dependency graph review in CI
NFR-08	Operational budget	All execution within free-tier limits; \$0.00 billed	Provider billing audit at milestone close

10.1 Where these numbers come from

Targets stated without derivation are guesses wearing a uniform. The reasoning behind the three latency numbers:

NFR-01, 180 seconds. This is the outer boundary of tolerable wait for a task the user is actively watching. Beyond roughly three minutes, attention departs even with live progress reporting. The eight-stage progress surface (§11.4) exists specifically as mitigation — it converts an opaque wait into an observable one, which materially extends tolerance without changing the underlying duration.

NFR-02, 800 ms. Search must feel interactive. The dominant cost is not FAISS retrieval, which is sub-10 ms against a warm in-memory index, but query embedding on CPU plus network round-trip. The budget allocates generously to embedding so that the target holds even on a cold model load.

NFR-03, 4.0 seconds. Dominated entirely by Gemini generation latency, over which the system has no control. The budget is set at a level achievable with Flash-class inference and is the reason a larger reasoning model was rejected in ADR-005 — the quality gain would not survive the latency cost.

COLD-START EXCLUSION

NFR-02 and NFR-03 govern steady-state request latency and explicitly exclude the first request against a repository after a process restart, which additionally incurs FAISS reconstruction from PostgreSQL. That cost is bounded and is documented in §16.3; treating it as an SLA violation would misattribute a deliberate architectural trade to a performance defect.

11. User Workflow and Product Design

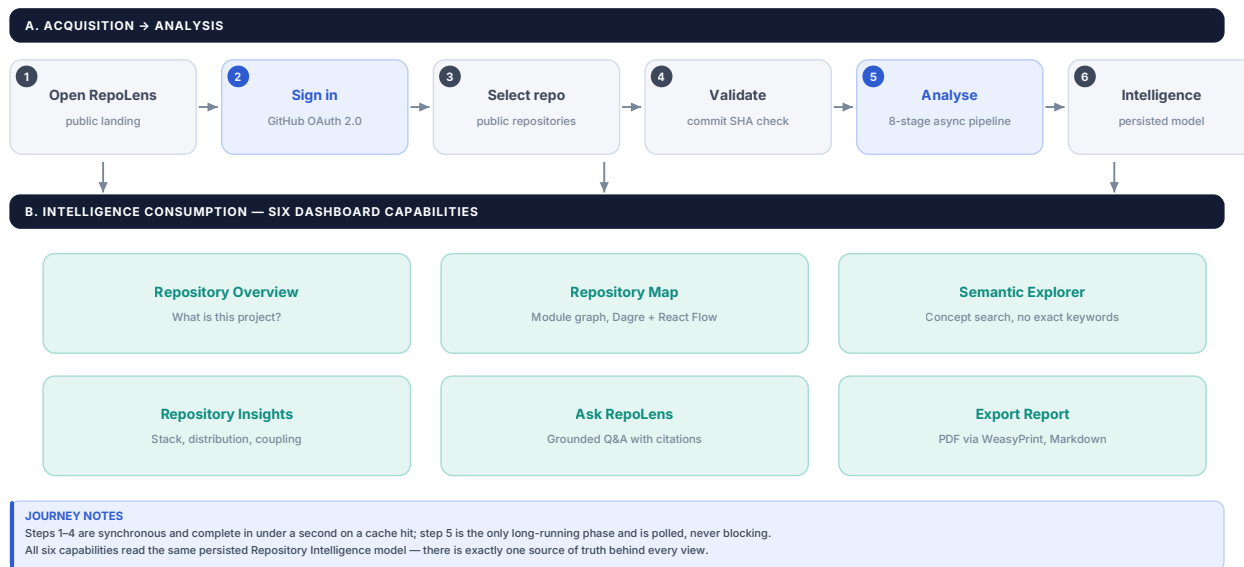


Figure 11.1 — End-to-end user journey. Acquisition and analysis are strictly sequential; the six consumption capabilities are peers that all read the same persisted model. The SHA check at step 4 is the branch that determines whether the user waits seconds or minutes.

11.1 Landing and authentication

The landing page explains the problem, previews the six capabilities, lists supported languages, and presents exactly one action: sign in with GitHub. There are no secondary conversion paths, because there is no tiering to convert between.

Authentication is exclusively GitHub OAuth 2.0. Passwords, custom account creation and third-party social login are omitted deliberately: every meaningful action in the product is already scoped to a GitHub identity, so a separate identity would add a trust boundary without adding capability.

ENGINEERING INSIGHT — RATE LIMIT ELEVATION

Authenticating clone and REST calls with the user's own token raises GitHub's limit from 60 requests/hour shared per IP to 5,000 requests/hour per user. This is load-bearing, not convenience: at the unauthenticated tier a handful of concurrent analyses would exhaust the platform's entire budget. The per-user token model makes rate capacity scale linearly with active users rather than being a fixed shared pool — the system's concurrency ceiling is therefore a property of adoption rather than of infrastructure.

11.2 Repository selection and version validation

The selection dashboard lists accessible public repositories with name, owner, description, primary language, size and last-updated timestamp. On selection the backend resolves the remote HEAD SHA and compares it against any stored analysis record.

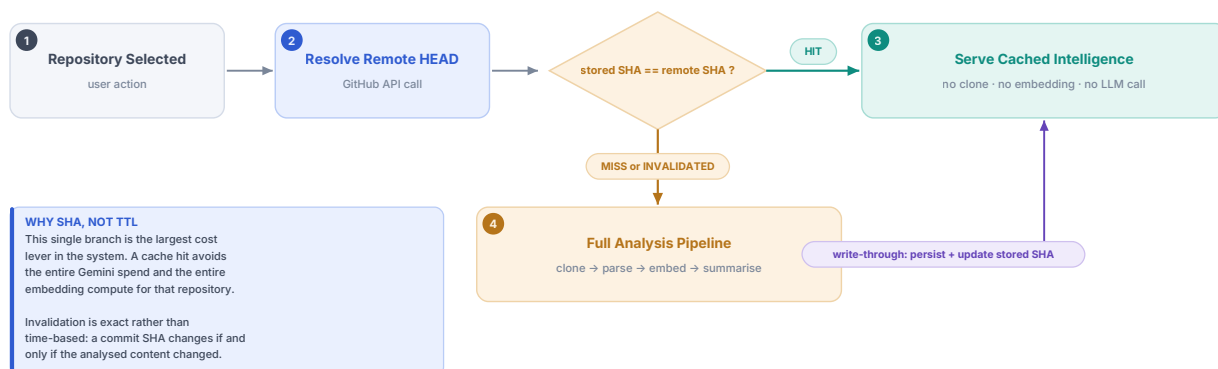
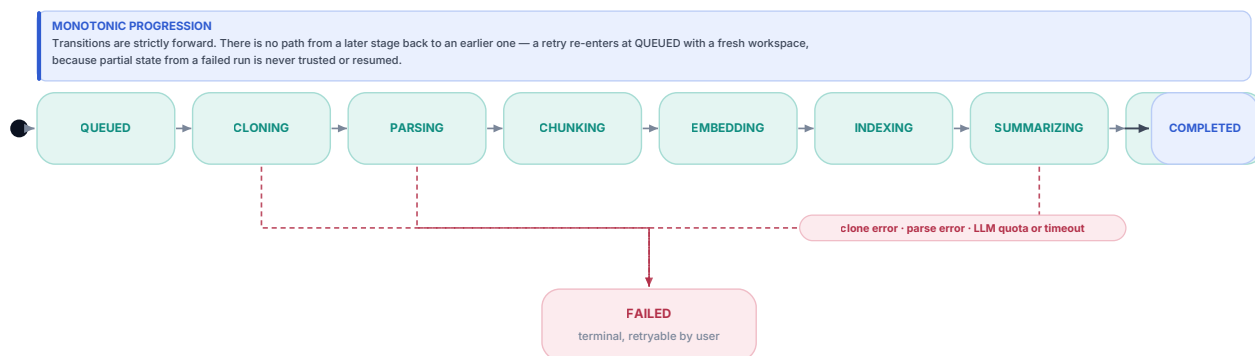


Figure 11.2 — SHA-gated cache validation. Invalidation is exact rather than time-based: a commit SHA changes if and only if the analysed content changed, which eliminates both stale-serve risk and the wasted recomputation a TTL policy would cause on unchanged repositories.

11.3 Asynchronous analysis

Analysis executes as a FastAPI background task. The client polls `GET /api/v1/analysis/{id}/status` every two seconds and renders one of eight named stages. Polling was chosen over server-sent events or websockets for a specific reason: the status value is a single enum column updated a handful of times per job, and a push channel would add connection lifecycle management, reconnection handling and a second transport to test — real cost for no user-perceptible gain at a two-second granularity.



The status field is a single enum column on repositories, read by the polling endpoint. No message broker is involved: at this concurrency a polled column is simpler and sufficient.

Figure 11.3 — Analysis state machine. Progression is strictly monotonic; there is no backward transition. A retry re-enters at QUEUED with a fresh workspace because partial state from a failed run is never trusted or resumed — resumption would require checkpointing every stage, which is unjustified complexity for a job measured in minutes.

11.4 The eight stages and what they mean to the user

#	Stage	What is happening	Why it is surfaced separately
1	Cloning repository	Shallow clone into an isolated workspace	Duration scales with repository size — the user's mental model of "why is this slow" starts here
2	Detecting technologies	Manifest and config inspection, language census	Fast; its completion signals the repository is valid and supported
3	Parsing source code	Tree-sitter AST construction per file	The first stage whose failure implies an unsupported or malformed input
4	Extracting metadata	Imports, exports, declarations, signatures	Separated from parsing so a partial-extraction failure is diagnosable
5	Detecting modules	Two-pass clustering plus LLM naming	The first stage involving an external API; isolates quota failures
6	Building intelligence	Chunking, embedding, indexing	Longest CPU-bound stage; users benefit from knowing progress continues
7	Generating insights	Map-Reduce summarisation	Second external-API stage; the most likely place to hit rate limits
8	Preparing dashboard	Graph layout, learning path, persistence commit	Signals imminent completion, which measurably reduces abandonment

11.5 The six capabilities

Capability 1 — Repository Overview

A zero-input answer to what is this project. Metrics header (primary language, language breakdown, framework tags, file count, module count, disk size), an executive summary synthesised by the reduce stage, and highlighted entry points such as `main.py`, `src/index.tsx`, `App.java`.

Capability 2 — Interactive Repository Map

The architecture at module granularity. Nodes carry a module name, category (Auth, Database, API, Utilities and similar), file count and stated responsibility; directed edges represent import-derived couplings. Clicking a node opens a drawer with file listings, exported symbols and internal dependencies. Layout is computed server-side by Dagre and rendered by React Flow.

DELIBERATE OMISSION

Individual files are excluded from the graph. A 3,000-file repository rendered at file granularity produces a hairball that restates the comprehension problem rather than solving it. File-level detail is reachable through the module drawer, which preserves access without sacrificing the overview.

Capability 3 — Semantic Explorer

Concept-driven exploration without exact keywords — where is JWT validation handled, show me database caching logic. Returns ranked chunks with their containing module, a relevance score, and a natural-language context line.

Capability 4 — Repository Insights

Structured observation cards: technology stack breakdown, module distribution across business logic versus infrastructure versus utilities, dependency flow and coupling concentrations, and identified structural patterns such as layered architecture or MVC.

Capability 5 — Ask RepoLens

Grounded conversational Q&A. Answers derive strictly from retrieved chunks and module metadata, with file and line citations attached. Below the 0.60 similarity floor the system returns "Insufficient evidence within repository source code to answer this query confidently" and makes no model call. Session state is stateless server-side: the client sends the last three turns with each request.

Capability 6 — Export Report

Server-rendered PDF via WeasyPrint from Jinja2 templates with print media rules, and GitHub-Flavored Markdown from a parallel template over the same model — content parity between formats is structural, not maintained by discipline.

11.6 Learning path and global search

The generated reading order is: `README → configuration → core auth and data models → primary business modules → API layer → utilities → tests`. The sequence is not arbitrary: it mirrors how an experienced engineer explores unfamiliar code, moving from stated intent through foundational structures before business logic, and treating tests as confirmatory reading rather than an entry point.

A global `Ctrl+K` palette, available on every dashboard route, opens a modal semantic explorer for context-preserving lookup.

11.7 Interface principles and error surfaces

Minimalist, high-contrast, developer-oriented, no decorative animation — visually closer to GitHub, Vercel and Linear than to consumer product design. The pre-analysis empty state carries a single call to action.

TABLE 11.2 — ERROR CONDITIONS AND USER-FACING TREATMENT

Condition	User-facing message strategy	Server-side handling
Repository exceeds 3,000 files	State the boundary plainly and suggest a smaller target	Rejected before clone; no workspace created
GitHub rate limit exceeded	Explain that the limit is temporary and indicate retry timing	Rate-limit headers logged; job marked failed with a distinct category
Unsupported primary language	Name the four supported languages explicitly	Partial analysis offered if a supported subset exists
Analysis timeout	Offer retry; no technical detail exposed	Full exception context logged with the analysis identifier for triage
Repository became private or was deleted	Serve last cached model with a banner noting the source is unavailable	Cached intelligence retained; re-analysis blocked

NEVER EXPOSE STACK TRACES

Stack traces are suppressed at the API boundary without exception. They leak file paths, dependency versions and internal structure, and they are useless to every persona in §7. Full context is written to the log stream with the analysis identifier, which is the only correlation key a user needs to quote in a report.

12. Acceptance Criteria and Edge Cases

12.1 Representative acceptance criteria

TABLE 12.1 — ACCEPTANCE CRITERIA IN GIVEN/WHEN/THEN FORM

Ref	Criterion
AC-03	Given a repository previously analysed at commit SHA <code>X</code> , when the user re-selects it and the remote HEAD is still <code>X</code> , then the cached model MUST be served and zero background jobs MUST be enqueued.
AC-04	Given any analysis job, when it terminates for any reason including an unhandled exception, then the temporary workspace MUST NOT exist on the filesystem afterwards.
AC-07	Given the same repository at the same commit, when module detection runs twice, then the resulting file-to-module assignment MUST be byte-identical.
AC-14	Given a query whose maximum retrieved similarity is below 0.60, when it is submitted, then the standardised fallback MUST be returned and the Gemini client MUST record zero invocations.
AC-15	Given a completed analysis, when both PDF and Markdown exports are generated, then every module, insight and learning-path step present in one MUST be present in the other.

12.2 Edge cases and handling

TABLE 12.2 — EDGE CASE REGISTER

Edge case	Handling	Rationale
Zero detectable modules — flat script collection	Synthesise a single Root module containing all files	Degraded output is more useful than a failed analysis; the map still renders
Mostly unsupported language with a supported minority	Analyse the supported subset; label coverage percentage prominently in the overview	Silent partial coverage would mislead; explicit partial coverage is honest and still useful
Very large generated file — for example a 20,000-line bundle	Apply a per-file chunk cap; flag as generated when marker heuristics match	Prevents one artefact from dominating the embedding space and polluting every search result
Repository deleted or made private post-analysis	Serve cached intelligence with an availability banner	The intelligence is our derived artefact and remains valid; only re-analysis is impossible
Fork of an already-analysed repository	Treated as a distinct record; no cross-fork deduplication	Forks diverge. Sharing intelligence across them would serve wrong data the moment they differ
Non-Latin paths and Unicode identifiers	Full UTF-8 throughout parsing, storage and rendering	ASCII assumptions fail silently and corrupt rather than error, which is the worst failure class
Binary files with source extensions	Encoding detection; skipped with a logged reason	Feeding binary content to a parser produces garbage chunks that degrade retrieval quality
Circular module dependencies	Rendered as bidirectional edges; Dagre layout tolerates cycles	Cycles are a real and interesting architectural property, not an error to suppress

12.3 Release scope

Category	Contents
MVP — v1.0	OAuth, SHA-gated selection, four-language Tree-sitter parsing, two-pass module detection, semantic chunking, embedding and FAISS retrieval, three-stage summarisation, grounded Q&A, interactive map, PDF and Markdown export, generated learning path
Deferred — §46	Additional language parsers, private repositories, multi-repository comparison, persistent chat history, IDE extension, collaborative annotation
Refused — §6.2	Code generation and editing, PR and CI automation, multi-agent orchestration

PART THREE

03 System Architecture

How the system is built and why it is built that way — layered structure, recorded decisions with their rejected alternatives, data flow, persistence, security, performance, observability and cost.

- 13 Architecture Overview
- 14 Architecture Decision Records
- 15 Data Flow and Processing Lifecycle
- 16 Persistence Strategy
- 17 Infrastructure and Deployment
- 18 Security and Threat Model
- 19 Performance Engineering
- 20 Observability
- 21 Failure and Disaster Recovery
- 22 Cost Model

13. Architecture Overview

RepoLens is a modular monolith. Every backend component executes in one FastAPI process behind strict internal package boundaries, without the network overhead, distributed tracing burden, or orchestration cost that service decomposition would impose on a two-engineer team operating inside a single free-tier hosting allowance.

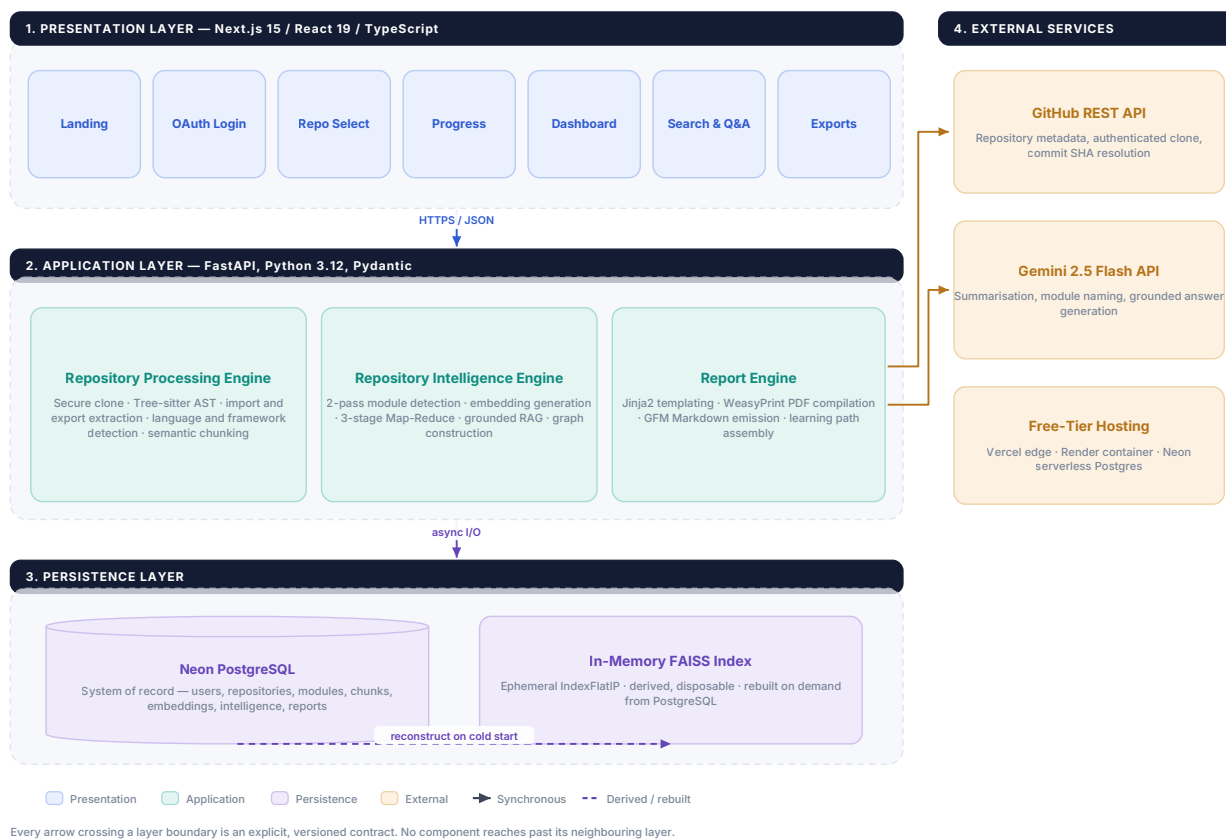


Figure 13.1 — Layered architecture. The application layer isolates three engines behind package boundaries, delivering most of the maintainability benefit of service separation at none of its operational cost. PostgreSQL is the sole system of record; FAISS is derived and disposable.

13.1 Component responsibilities

TABLE 13.1 — COMPONENT RESPONSIBILITY MATRIX

Component	Technology	Responsibilities
Frontend application	Next.js 15, React 19, TypeScript	Authentication UI, repository selection, dashboard composition, React Flow map, command palette, export triggers
API gateway	FastAPI, Python 3.12, Pydantic	OAuth handling, routing, request and response validation, background task orchestration, session verification
Repository Processing Engine	Tree-sitter, Git, tempfile	Shallow cloning, tree traversal, AST construction, import and export extraction, language and framework detection, semantic chunking
Repository Intelligence Engine	bge-small-en-v1.5, FAISS, Gemini 2.5 Flash	Embedding generation, index construction, two-pass module detection, Map-Reduce summarisation, grounded retrieval and answering
Report Engine	Jinja2, WeasyPrint	Template rendering, PDF pagination, Markdown emission, learning path assembly
Relational store	Neon PostgreSQL, SQLAlchemy	System of record for users, repositories, modules, chunks, embeddings, intelligence and reports
Vector index	FAISS IndexFlatIP, in-memory	Similarity search over chunk embeddings; rebuilt on demand from the relational store

13.2 Why modular monolith rather than microservices

This deserves explicit treatment because it is the fork where an unexamined default in either direction produces a worse system. Microservices would introduce inter-service authentication, network-boundary error handling, distributed tracing, and multiplied hosting instances — none of which purchase anything for two engineers sharing one deployment budget. The failure mode microservices exist to prevent, many teams colliding in an undifferentiated codebase, does not occur at this team size.

Equally, an unstructured monolith would produce precisely the tangled, untestable codebase RepoLens exists to help developers escape. The modular qualifier is doing real work: the three engines communicate through explicit interfaces, have no circular imports, and are independently unit-testable. NFR-07 makes this a verified property rather than an aspiration. Full reasoning is in ADR-001.

14. Architecture Decision Records

An ADR records not only what was decided but what was seriously considered and rejected, so a future maintainer inherits the reasoning rather than only the conclusion. Eight decisions are recorded; Appendix C consolidates them.

ADR-001

Modular monolith over microservice decomposition

ACCEPTED

CONTEXT

Three logically distinct backend concerns — processing, intelligence, reporting — could plausibly be independently deployed services.

DECISION

Deploy a single FastAPI process with strict internal package boundaries between the three engines.

ALTERNATIVES CONSIDERED

(a) Full microservice decomposition with per-engine deployment. (b) Serverless functions per pipeline stage.

WHY REJECTED

(a) Requires inter-service auth, network error handling, and multiple hosting instances; the free tier supports a limited number of always-on services, so splitting either exceeds the allowance or forces cold-start-prone functions for at least one engine. (b) Cold-start latency directly threatens NFR-01 to NFR-03, and the in-memory FAISS model depends on process-level memory residency that serverless invocations cannot provide.

CONSEQUENCES

The three engines share a process and a deployment lifecycle. Internal discipline substitutes for enforced separation, and NFR-07 makes that discipline auditable. If the system ever needed independent scaling, the package boundaries are already the natural extraction seams.

ADR-002

PostgreSQL as system of record, FAISS as ephemeral derived index

ACCEPTED

CONTEXT

Similarity search needs a fast in-memory index, but free-tier containers restart and sleep, discarding local disk and memory.

DECISION

Persist every embedding durably in PostgreSQL rows. Treat FAISS as a disposable acceleration structure rebuilt from PostgreSQL on startup or first request.

ALTERNATIVES CONSIDERED

(a) A managed vector database — Pinecone, Weaviate Cloud, Qdrant Cloud. (b) Persisting the FAISS index file to container disk.

WHY REJECTED

(a) Free tiers impose vector-count or query-volume ceilings inconsistent with the zero-cost requirement, and add a third external dependency alongside GitHub and Gemini. (b) Free-tier disk is not guaranteed across restarts, making a disk-resident index an unreliable source of truth — precisely the failure this decision exists to prevent.

CONSEQUENCES

A bounded rebuild cost on cold start in exchange for zero vector-loss risk. The index becomes a pure function of relational state, which also makes horizontal scaling a configuration change rather than an architectural one, since instances need no shared index coordination.

ADR-003

Deterministic structural extraction before generative summarisation

ACCEPTED

CONTEXT

Module boundaries could be inferred directly by a model reading the file tree and sampled contents, avoiding a dedicated clustering algorithm entirely.

DECISION

Build a deterministic two-pass detector over directory topology and import cohesion. Restrict the model to validating and naming pre-computed candidates.

ALTERNATIVES CONSIDERED

Direct LLM structural inference from file listings and sampled file contents.

WHY REJECTED

Model-only inference produced architecturally plausible but incorrect groupings on repositories with unconventional layouts — files clustered by naming similarity rather than actual import coupling. That failure is undetectable without doing the manual verification the tool exists to eliminate, and it contradicts constraint DG-1.

CONSEQUENCES

Additional engineering in §25. In exchange, boundaries are reproducible across runs and traceable to concrete import evidence, which is what makes the output auditable rather than merely persuasive.

ADR-004

Similarity-threshold gate ahead of generation, not output filtering

ACCEPTED

CONTEXT

Conventional RAG generates from whatever context retrieval returns, regardless of retrieval quality.

DECISION

Enforce a hard 0.60 cosine floor on the top retrieved chunk. Below it, abort before the model is called and return a standardised response.

ALTERNATIVES CONSIDERED

(a) Always generate and rely on prompt instructions to self-regulate. (b) An adaptive or per-repository threshold.

WHY REJECTED

(a) Prompted self-regulation reduces but does not eliminate confident fabrication, particularly for questions a model can answer plausibly from general programming knowledge without the repository at all — the exact case the gate must catch. (b) No calibration signal is available at inference time to tune per repository; the parameter would add complexity without a demonstrated accuracy gain.

CONSEQUENCES

Some legitimate but obliquely phrased questions receive a false negative. That trade is chosen deliberately: a false "I don't know" is recoverable by rephrasing, a false confident answer is not recoverable at all.

ADR-005

Hosted Gemini 2.5 Flash over a self-hosted open-weight model

ACCEPTED

CONTEXT

Summarisation and answering could run against a hosted API or a locally-hosted open-weight model.

DECISION

Use the Gemini 2.5 Flash API for all generative work.

ALTERNATIVES CONSIDERED

Quantised open-weight models served locally in the backend container.

WHY REJECTED

Free-tier compute cannot host a generation-quality model at acceptable latency alongside the rest of the process. CPU-only inference at the context lengths Map-Reduce requires would exceed NFR-01 to NFR-03 by a wide margin, and would compete for the same cores the embedding pipeline needs.

CONSEQUENCES

A hard dependency on external API availability and free-tier quota, tracked in §45. Mitigated structurally by the failure asymmetry in §23: Tier 1 output is persisted before any generative stage runs, so quota exhaustion degrades the product rather than failing it.

ADR-006

Tree-sitter over native per-language parsers

ACCEPTED

CONTEXT

Each supported language ships an official parser — Python's `ast`, a Java front-end, the TypeScript compiler API.

DECISION

Use Tree-sitter grammars behind one unified traversal API for all four languages.

ALTERNATIVES CONSIDERED

Four native parser integrations, each with its own node schema and traversal code.

WHY REJECTED

Four independent code paths quadruple the surface a two-engineer team must maintain and test, and invite inconsistent extraction quality across languages. Tree-sitter's uniform query API is worth more than per-language fidelity we do not need.

CONSEQUENCES

Analysis is purely syntactic — no type resolution, no cross-file symbol resolution. Every downstream component is designed around that limit, which is why cohesion is computed from import edges rather than resolved call targets (§25.2).

ADR-007

Polled status column over a push channel

ACCEPTED

CONTEXT

Analysis progress must reach the client during a multi-minute background job.

DECISION

Expose a status endpoint backed by a single enum column; the client polls every two seconds.

ALTERNATIVES CONSIDERED

Server-sent events, or a websocket channel carrying stage transitions.

WHY REJECTED

Status changes a handful of times per job. A push channel adds connection lifecycle, reconnection logic, and a second transport to test, for no perceptible gain at two-second granularity. It would also complicate the stateless backend model that makes horizontal scaling trivial.

CONSEQUENCES

Up to two seconds of staleness in the progress display, which is imperceptible against stage durations measured in tens of seconds.

ADR-008

Normalised inner product over Euclidean distance for retrieval

ACCEPTED

CONTEXT

FAISS supports several distance metrics; the choice affects both index type and retrieval quality.

DECISION

L2-normalise all embeddings and use `IndexFlatIP`, which on unit vectors is mathematically equivalent to cosine similarity.

ALTERNATIVES CONSIDERED

`IndexFlatL2` over unnormalised vectors.

WHY REJECTED

The BGE model family is trained and benchmarked under cosine-similarity objectives; using a metric the embedding space was not optimised for degrades retrieval for no benefit.

CONSEQUENCES

An explicit normalisation step is mandatory at both index time and query time. Omitting it at either point silently corrupts scores, which is why it is an assertion in the test suite rather than a convention.

15. Data Flow and Processing Lifecycle

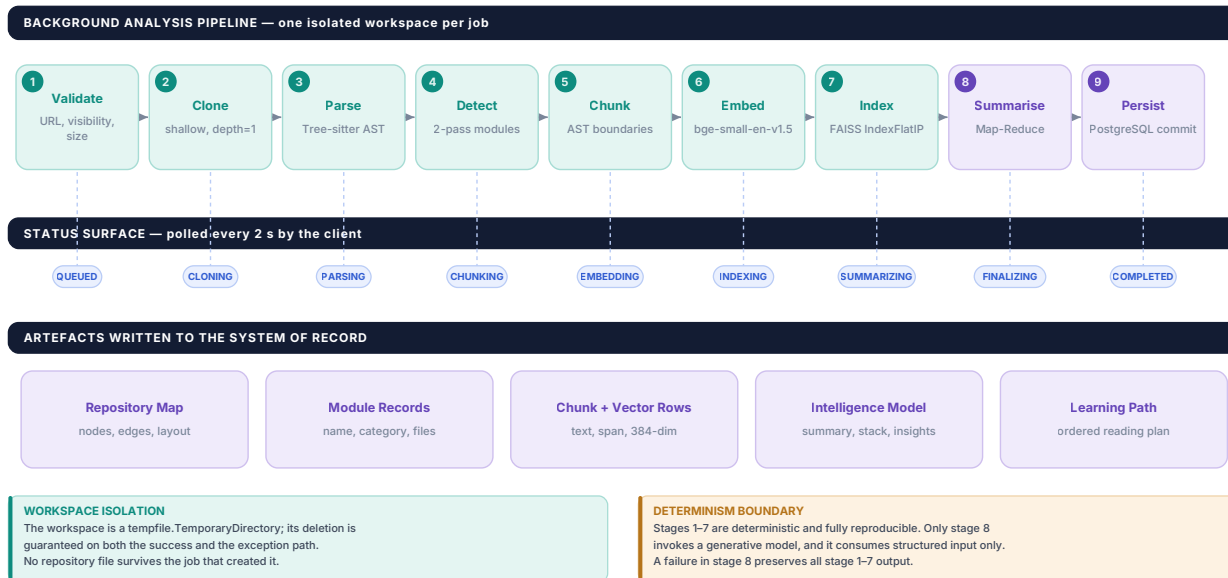


Figure semantics: solid arrows are data movement; dotted verticals project each pipeline stage onto the status value the client observes.

Figure 15.1 — The analysis pipeline end to end, with the status projection the client observes and the artefacts written to the system of record. Stages 1 through 7 are deterministic; only stage 8 invokes a generative model, and it consumes structured input exclusively.

15.1 Lifecycle invariants

Four properties hold for every analysis job, and each is asserted in the test suite rather than assumed:

- **Workspace isolation.** Each job receives a uniquely-prefixed temporary directory. No shared mutable filesystem state exists between concurrent jobs.
- **Read-only access.** Repository content is opened for reading and parsed as text. Nothing in the cloned tree is executed at any point.
- **Guaranteed cleanup.** The workspace is destroyed on both the success and the exception path, enforced by a context manager rather than by explicit cleanup calls that an early return could bypass.
- **Durable commit last.** Intelligence is persisted as the final pipeline step. A crash before that point loses work but never leaves a partially-written model that a later request could read as complete.

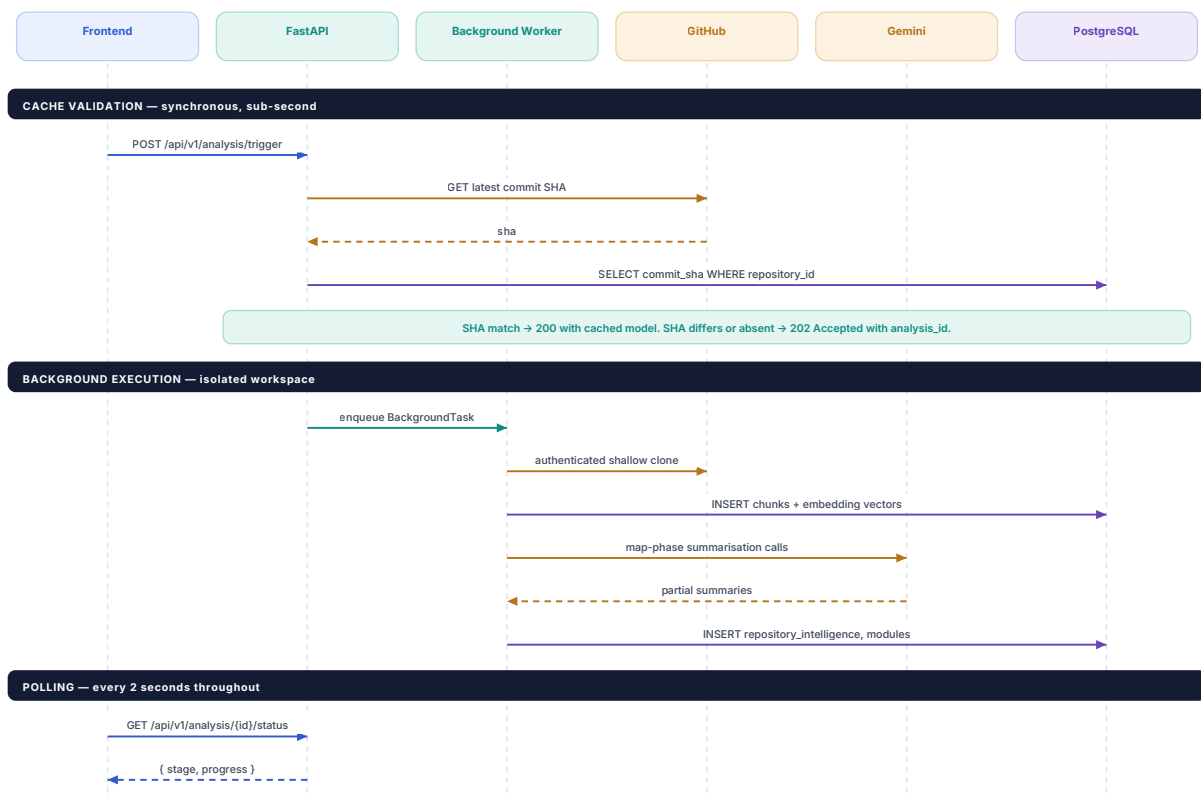


Figure 15.2 — Analysis sequence, showing the synchronous cache-validation phase, the asynchronous background execution, and the polling loop that runs concurrently with both. Note that the trigger endpoint returns before any expensive work begins.

16. Persistence Strategy

RepoLens stores Repository Intelligence, not repositories. The cloned codebase is an ephemeral processing input; once parsing, chunking, embedding and generation complete, the working directory is destroyed.

16.1 PostgreSQL as sole system of record

Chunk text, metadata and 384-dimensional embedding vectors are stored directly in PostgreSQL rows — not referenced by a pointer into a second store. This is a deliberate refusal to introduce a second storage system: native array and JSONB column types hold embeddings alongside their originating text, keeping the system of record genuinely singular and eliminating an entire class of cross-store consistency bugs.

16.2 FAISS as derived acceleration

On startup, or on the first request against a repository not resident in the current process, embeddings are read from `repository_chunks` and batch-inserted into an in-memory `IndexFlatIP`. Retrieval is then sub-10 ms for the remainder of the process lifetime.

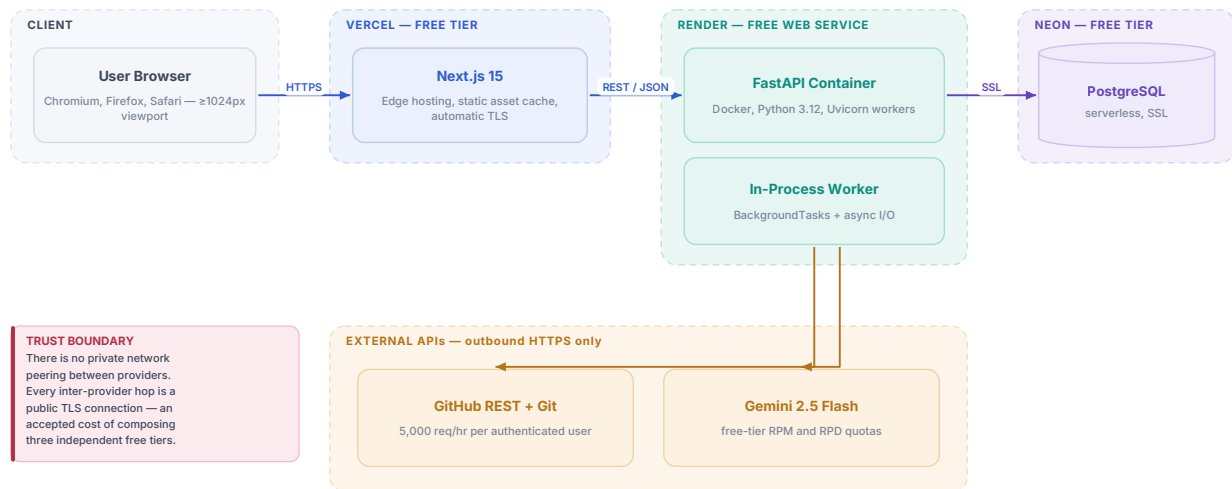
16.3 The trade this buys

WHY REBUILDING BEATS PERSISTING

A container restart, sleep-wake cycle, or scale-out event never risks vector loss, because the index is a pure and cheaply recomputable function of relational state. The cost is a bounded rebuild on cold start; the benefit is that an entire category of durability incident cannot occur. For a system on ephemeral free-tier infrastructure, that is the correct direction to take the trade.

A second consequence is architectural rather than operational: because no instance owns authoritative index state, multiple backend instances could each hold their own in-memory index with no coordination protocol between them. Horizontal scaling is therefore a configuration change, which is why §19.2 can describe it in a paragraph rather than a chapter.

17. Infrastructure and Deployment



Deployment topology. Each provider is independently replaceable; no provider-proprietary primitive is used anywhere in the system.

Figure 17.1 — Deployment topology across three independent free-tier providers. Every inter-provider hop is a public TLS connection; there is no private peering, which is the accepted cost of composing independent free tiers rather than adopting one vendor's integrated stack.

17.1 Provider allocation

Layer	Provider	Responsibility	Why this provider
Frontend	Vercel	Next.js hosting, edge caching, automatic TLS	First-class Next.js support; Git-triggered deploys need no separate CI stage
Backend	Render	Dockerised FastAPI container	Supports long-running always-on processes, which serverless platforms do not — required for in-memory FAISS residency
Database	Neon	Serverless PostgreSQL with SSL	Native array and JSONB support lets one store hold both relational and vector data
Generation	Google Gemini	Flash-class inference	Large context window at a free tier that tolerates Map-Reduce call volumes

17.2 Continuous delivery

Both Vercel and Render support Git-triggered deployment natively, so no separate deployment orchestration service is required. GitHub Actions is reserved exclusively for pre-merge quality gates (§40.1) rather than for deploying — a division that keeps the deployment path short and the test path strict.

17.3 Free-tier constraints and their handling

Constraint	Provider	Handling
Container sleeps after inactivity	Render	Absorbed by the rebuild-on-demand design (§16.2). A lightweight health endpoint may be pinged during evaluation windows to reduce sleep frequency.
Build minutes and compute hours	Vercel, Render	Well within limits at evaluation-period usage; monitored at each milestone close.
Storage allocation	Neon	Bounded by the 3,000-file envelope; per-repository footprint quantified in §22.1.
API request quotas	GitHub, Gemini	Per-user OAuth elevation for GitHub (§11.1); Map-Reduce batching bounds Gemini call count to module count rather than file count.

18. Security Architecture and Threat Model

RepoLens ingests arbitrary, attacker-controllable content from the public internet and processes it in our execution context. That single fact determines the entire security posture: the boundary between untrusted input and trusted execution must be explicit, narrow, and enforced structurally rather than by convention.

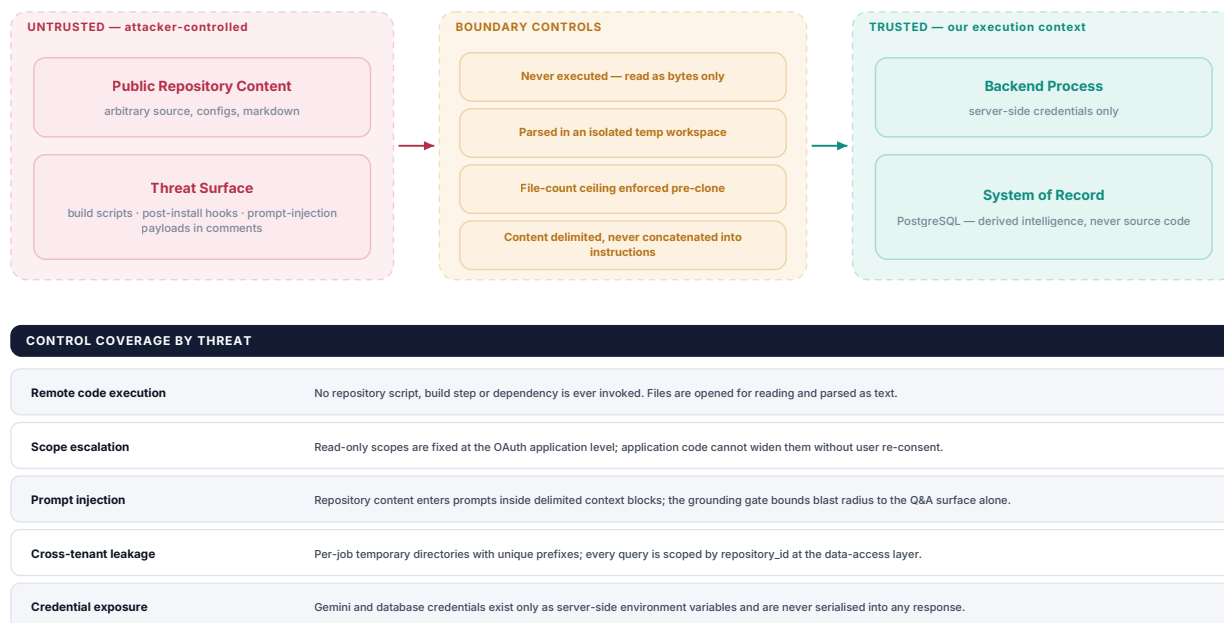


Figure 18.1 — Trust boundary and control coverage. Repository content never crosses into the trusted context as anything other than inert text; the controls in the centre column are the only permitted transit path.

18.1 Read-only access model

OAuth scopes are limited to `read:user` and public repository read access. There is no write scope, no commit capability, and no pull-request permission at any point in the request path. Critically, this is enforced at the OAuth application registration level rather than in application code — a compromised backend cannot widen its own grant, because widening requires user re-consent through GitHub's own interface.

ENGINEERING INSIGHT — WHY SCOPE PLACEMENT MATTERS

There is a meaningful difference between "our code does not call write endpoints" and "our token cannot call write endpoints." The former is a property of the current implementation and can regress in any commit. The latter is a property of the credential itself and survives arbitrary application compromise. NG2 in §6.2 — refusing PR and CI automation — exists partly to keep this property intact, and is an example of the security posture constraining the feature set rather than the reverse.

18.2 No code execution, ever

Cloned files are opened for reading and parsed as text. No repository script, build system, package manifest installation, or post-install hook is invoked at any point. This is the single most important control in the system: without it, a malicious public repository would turn RepoLens into a remote code execution service, and every other control would be irrelevant.

IMPLEMENTATION WARNING

Several conveniences would violate this control and must be refused during implementation: running `npm install` or `pip install` to resolve dependency versions; invoking a language's own parser via a subprocess that imports the target file; executing test discovery to enumerate test files; using any library that lazily evaluates repository configuration files. If a capability requires executing repository content, the capability is out of scope — this supersedes any product argument.

18.3 Workspace isolation

Each job parses inside a uniquely-prefixed OS temporary directory under `/tmp/repoLens_*` with destruction guaranteed by context manager exit. Concurrent jobs cannot observe or interfere with one another's filesystem state, and no residue survives a completed job.

18.4 Threat model

TABLE 18.1 — THREATS, VECTORS, AND CONTROLS

Threat	Vector	Severity	Control
Remote code execution	Malicious build script, setup hook, or dependency post-install in a cloned repository	Critical	No repository content is ever executed (§18.2). Files are read as bytes and parsed syntactically.
OAuth scope escalation	Compromised backend attempts a write operation with the user's token	High	Read-only scopes fixed at the OAuth application level; escalation requires user re-consent and cannot originate server-side.
Prompt injection	Repository comments or documentation crafted to redirect model behaviour	Medium	Content enters prompts inside delimited context blocks, never concatenated into instruction position (§31). The grounding gate bounds blast radius to the Q&A surface; Tier 1 structural data is unreachable by injected text.
Cross-tenant data leakage	One job reads another's workspace, or a query returns another repository's chunks	High	Per-job unique temporary prefixes; every retrieval and every query is scoped by <code>repository_id</code> at the data-access layer, not at the presentation layer.
Credential exposure	Gemini key or database URL reaching the client	High	All external credentials exist only as server-side environment variables and are excluded from every response schema by construction — no response model contains a field that could carry one.
Denial of service	Adversarially expensive repository — deeply nested generated files, pathological AST depth	Medium	File-count ceiling enforced before clone; per-file chunk caps; parse timeouts per file.
Session hijacking	Stolen JWT reused from another origin	Medium	Short-lived signed tokens over TLS; no sensitive write capability exists behind the session to abuse even if replayed.
Storage exhaustion	Accumulated clone residue filling container disk	Low	Context-manager-guaranteed deletion (§18.3), asserted on exception paths by NFR-04.

18.5 Privacy posture

Only public repository content is processed — content the owner has already made publicly accessible. User data is limited to the GitHub profile fields required for authentication: numeric identifier, username, avatar URL. No email address, no private repository listing, no organisational membership is requested or stored. This is not merely a policy statement; it is a direct consequence of the minimal scope in §18.1, which makes the additional data unavailable to request.

19. Performance Engineering

19.1 Latency budget

TABLE 19.1 — COST CONTRIBUTION AND CONTROL LEVER PER STAGE

Stage	Scaling behaviour	Bound	Primary lever
Cloning	Linear in repository bytes	Network	Shallow clone at <code>--depth=1</code> ; full history is never needed and often dominates transfer size
Tree-sitter parsing	Near-linear in file count	CPU	C-based parser; rarely the bottleneck. Per-file timeout guards pathological inputs
Metadata extraction	Linear in AST nodes	CPU	Single-pass traversal; no re-parsing between extraction targets
Chunking	Linear in AST nodes	CPU	Reuses the parse tree already in memory rather than re-walking the file
Embedding	Linear in chunk count	CPU	Batched inference amortises per-call model overhead; the dominant CPU cost in the pipeline
Index construction	Linear in chunk count	Memory	Single batched <code>faiss.add()</code> rather than per-vector insertion
Map-Reduce summarisation	Linear in module count	External API	Parallel map calls; the wall-clock cost tracks group count, not repository size
Persistence	Linear in row count	Network + DB	Bulk insert of chunk rows in a single transaction

WHERE THE TIME ACTUALLY GOES

For a mid-sized repository the two dominant costs are embedding inference (CPU, ours) and Map-Reduce generation (external API, not ours). This split matters operationally: the first is optimisable by batching and model choice, the second is optimisable only by reducing call count. It is the reason §22.2 identifies SHA-gated caching as the single largest lever — it eliminates both costs simultaneously for the repeat case.

19.2 Scalability strategy

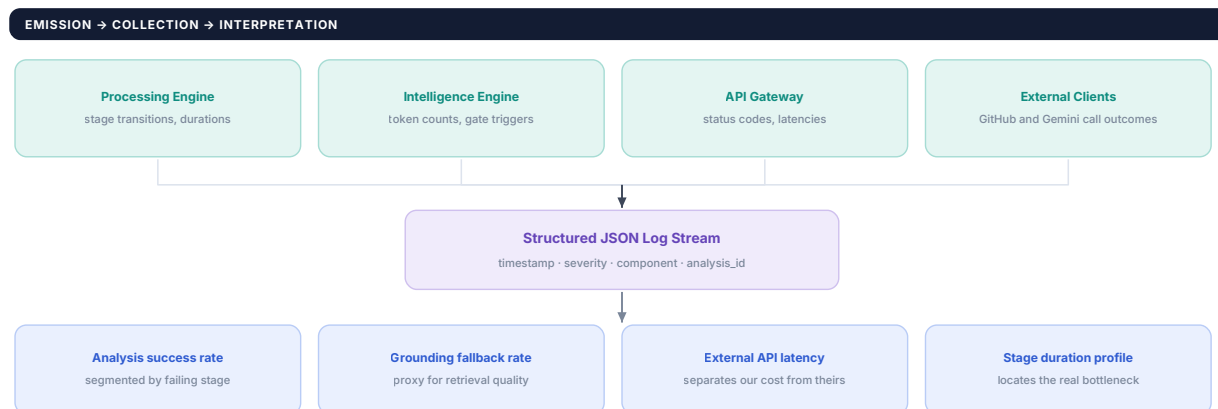
Scalability here is deliberately bounded, but the choices made are the same ones that would permit unbounded scaling if the budget changed:

- **Stateless request handling.** No request depends on process-local state except the FAISS index, which is derived and independently reconstructable per instance. Adding backend instances therefore requires no coordination protocol.
- **Single source of truth.** Because PostgreSQL is authoritative, instances cannot diverge. Cache coherency, the usual hard problem in horizontal scaling, does not arise.
- **Extraction seams already present.** The three engines have no circular imports (NFR-07), so any one could be lifted into a separate service without restructuring the others.
- **The known upgrade path.** In-process `BackgroundTasks` is appropriate at current concurrency. The natural successor — a dedicated queue with Celery and Redis — is named in §46 rather than built now, on the principle that infrastructure is added when a bottleneck is measured, not when it is imagined.

ACKNOWLEDGED LIMITATION

Tree-sitter parsing and embedding inference are synchronous and CPU-bound. `async` yields no concurrency for them — a long analysis will occupy a worker thread and reduce the throughput available to other requests on the same instance. This is accepted at the target concurrency and is stated here rather than discovered later. The mitigation, when needed, is a worker process pool, not more `async`.

20. Observability



No commercial APM is used — a paid or account-bound observability vendor is disproportionate at this scale. Structured logs into the platform log viewer are sufficient for a two-engineer team on a single instance.

Figure 20.1 — Observability path from emission through collection to the four signals that actually drive engineering decisions. No commercial APM is used; structured logs into the platform log viewer are sufficient at this scale and add no account dependency.

20.1 Logging standard

All components emit structured JSON with a fixed field set: timestamp, severity, component, and — where applicable — `analysis_id`. The correlation identifier is mandatory on every log line emitted inside a background job, because without it interleaved concurrent jobs are unreadable.

Category	Emitted by	Purpose
Stage transitions	Processing engine	Backs the status endpoint and provides post-hoc duration profiling per stage
External call outcomes	GitHub and Gemini clients	Latency, status, and rate-limit headers — separates our latency from theirs when an NFR is missed
Grounding gate triggers	RAG module	Every fallback logged with query and top score; the aggregate is a direct quality signal for retrieval
Exceptions	All components	Full server-side context, never surfaced to the client (\$11.7)
Token accounting	Intelligence engine	Per-stage token counts, which is how quota exhaustion is predicted rather than discovered

20.2 The four signals that matter

- **Analysis success rate, segmented by failing stage.** An aggregate success rate is nearly useless; the segmentation is what identifies which stage is least reliable and therefore where engineering effort belongs.
- **Grounding fallback rate.** A proxy for retrieval quality. An unexpected rise indicates a chunking or embedding regression, not a user behaviour change — this signal has caught more real defects in similar systems than direct retrieval metrics, because it is user-triggered rather than synthetic.
- **External API latency percentiles.** Attributes NFR violations correctly. Without this, a slow Gemini day looks identical to a performance regression in our own code.

- **Per-stage duration distribution.** Locates the real bottleneck rather than the assumed one, which is the prerequisite for any optimisation work being worthwhile.

DEVELOPER NOTE — DELIBERATE OMISSION

No commercial observability vendor is integrated. Free tiers exist, but each adds an account dependency, a data-egress consideration, and a vendor SDK to maintain, which is disproportionate for a single-instance deployment read by two engineers. The named upgrade path if the project outgrows this is self-hosted Grafana with Loki — recorded in §46 so the decision is inherited rather than rediscovered.

21. Failure and Disaster Recovery

21.1 Failure handling by component

TABLE 21.1 — FAILURE MODES AND RECOVERY BEHAVIOUR

Component	Failure mode	Behaviour and recovery
Clone	Network failure, repository deleted mid-operation, visibility changed	Job marked FAILED with a specific error category. Workspace cleanup still guaranteed. User sees a plain-language banner and may retry.
Parser	Malformed file, encoding failure, per-file timeout	Individual file skipped with a logged reason; analysis continues. A single bad file must not fail a 3,000-file job.
Gemini API	Rate limit, transient error, timeout	Bounded exponential-backoff retry. On exhaustion the job fails at the generative stage, but all Tier 1 output is already persisted and remains queryable.
Backend crash mid-job	Unhandled exception, container restart, OOM	Status remains at its last committed stage. The client surfaces a stalled state after a timeout threshold and offers retry rather than polling indefinitely.
Database connectivity	Neon connection interruption	SQLAlchemy pool reconnection; in-flight requests fail with a retryable error rather than a raw connection exception.
FAISS memory pressure	Index too large for container memory	Bounded by the file-count ceiling. If exceeded, retrieval degrades to per-repository index eviction rather than process failure.

21.2 Degradation, not failure

THE FAILURE ASYMMETRY

Because the deterministic tier completes and persists before any generative stage runs, a total Gemini outage leaves the repository map, the module list, the file inventory, and semantic search fully operational. Only summaries, insights and Q&A are unavailable. This is not a happy accident — the ordering of the pipeline in §15 was chosen partly to produce it, and it is the single strongest argument for the two-tier design beyond correctness.

21.3 Disaster recovery posture

The posture is intentionally minimal because the architecture removes the need for most of it. PostgreSQL is the only durable store, and Neon's managed tier provides its own storage durability. The system therefore owns no storage layer requiring a bespoke backup regime.

The only unrecoverable state in the entire system is the ephemeral workspace during an active job — and by design that workspace holds nothing of lasting value. Its total loss means one analysis must be re-triggered, with zero effect on any completed analysis. A system whose worst-case data-loss event is "one job must be re-run" does not need a disaster recovery plan; it needs the property that produces that outcome, which §16 documents.

22. Cost Model

22.1 Free-tier budget and headroom

TABLE 22.1 — RESOURCE CONSUMPTION AGAINST FREE-TIER ALLOWANCES

Service	Constraining limit	Our usage pattern	Headroom
Vercel	Bandwidth and build minutes	Static frontend, evaluation-period traffic	Substantial
Render	Monthly service hours; sleeps when idle	One container, intermittent active use	Sufficient; sleep absorbed by \$16.2
Neon	Storage allocation	Chunk text plus 384-dim vectors per repository	Dozens of repositories — see calculation below
GitHub API	5,000 requests/hour per user	A handful of calls per analysis	Very substantial
Gemini API	Requests per minute and per day	Bounded by module count, not file count	Managed by batching; monitored per \$20.2

Storage estimate, worked

A representative 1,500-file repository yields roughly 6,000 chunks at about four chunks per file, consistent with the 300–500 token target in §26. Each chunk stores its text — typically a few hundred bytes to low kilobytes — plus a 384-dimensional float vector at 4 bytes per dimension, or 1,536 bytes. The vector component alone is therefore approximately 9 MB per repository; with text and metadata the total lands in the low tens of megabytes. Neon's free allocation accommodates dozens of analysed repositories concurrently, comfortably exceeding what evaluation requires.

22.2 Cost levers actually employed

- **SHA-gated caching (§11.2).** The dominant lever by a wide margin. An unchanged repository incurs zero Gemini cost and zero embedding compute on every subsequent visit.
- **Map-Reduce batching (§28).** Bounds Gemini call count to a function of module count rather than file count, which is what keeps a 3,000-file repository inside a per-minute request quota.
- **Local CPU embedding.** Eliminates per-embedding API cost entirely. `sentence-transformers` running in-process converts a recurring variable cost into a fixed compute cost already paid for.
- **Shallow cloning.** Reduces both transfer time and peak disk, which matters on a container with a modest ephemeral disk allocation.

COST AS AN ARCHITECTURAL INPUT

Every one of these levers is also a performance lever, which is not coincidental. On free-tier infrastructure, cost and latency are usually the same quantity measured differently — work not done is both free and fast. This is why §19 and §22 reference the same mechanisms rather than describing separate optimisation programmes.

PART FOUR

04

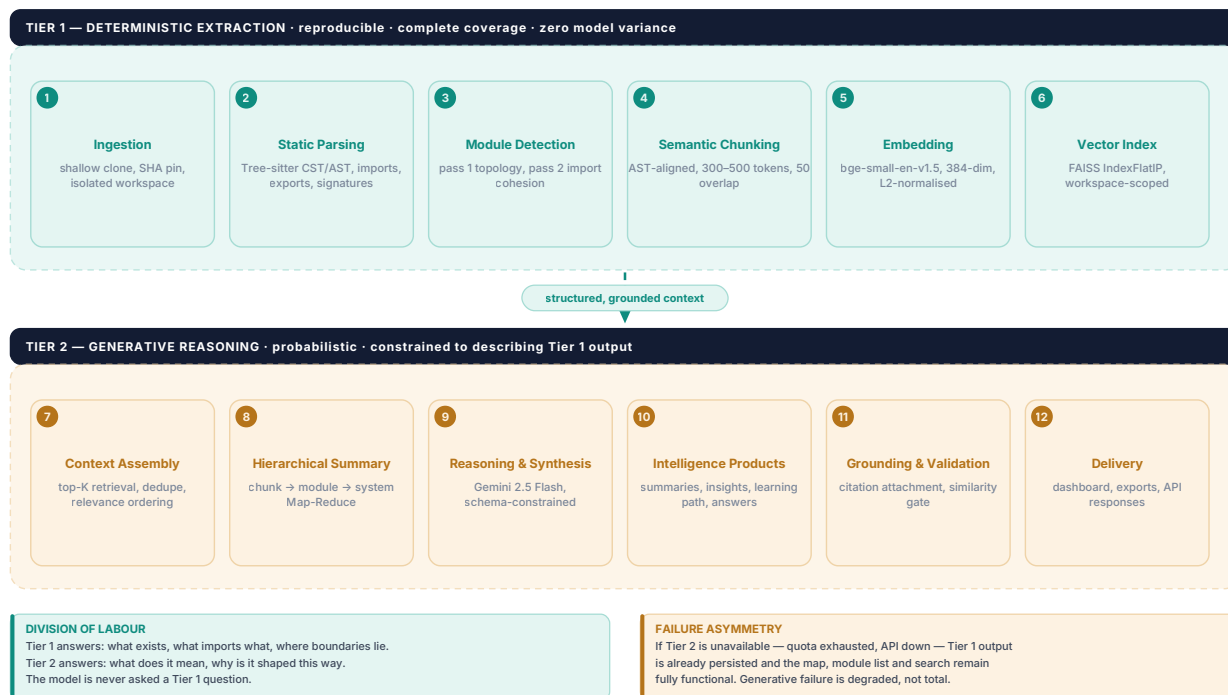
AI Engineering

The intelligence pipeline in full: the determinism boundary, static analysis, module detection, chunking, embeddings, hierarchical summarisation, and the grounding controls that make generated output trustworthy.

- 23 Two-Tier Intelligence Pipeline
- 24 Static Analysis with Tree-sitter
- 25 Module Detection Algorithm
- 26 Semantic Chunking
- 27 Embeddings and Vector Indexing
- 28 Map-Reduce Summarisation
- 29 Grounded RAG and Hallucination Control
- 30 Repository Map Layout Engine
- 31 Prompt Engineering Standards

23. Two-Tier Intelligence Pipeline

The system separates deterministic extraction from generative reasoning absolutely. Static parsing identifies factual structure — files, AST nodes, imports, declarations. Generation synthesises natural-language explanation from that structure and is never the origin of a structural claim.



This separation is the single most consequential decision in the system; ADR-003 records the alternative that was rejected to arrive at it.

Figure 23.1 — The two-tier pipeline. Tier 1 produces complete, reproducible coverage of factual structure; Tier 2 operates exclusively against Tier 1 output, never against raw repository content. The boundary between them is the architectural expression of constraint DG-1.

23.1 What belongs in which tier

Question	Tier	Why
Which files exist?	1	Enumerable. A model adds only the possibility of error.
What does file A import?	1	Derivable from the AST with certainty.
Which files form a module?	1	Computable from topology and import cohesion (§25). ADR-003 records why this must not be inferred.
What is this module called, in human terms?	2	Naming is genuinely a language task. The membership is already fixed.
What is this module responsible for?	2	Requires reading intent from code, which is synthesis, not extraction.
What architectural pattern does this resemble?	2	A judgement over structure that Tier 1 already established.
Where should a newcomer start reading?	2	A recommendation, grounded in Tier 1's dependency ordering.

THE TEST TO APPLY DURING IMPLEMENTATION

If a proposed model call would produce output that a parser could have produced, the call is wrong and the parser is missing. Every regression against DG-1 begins as a reasonable-sounding shortcut of exactly this shape.

24. Static Analysis with Tree-sitter

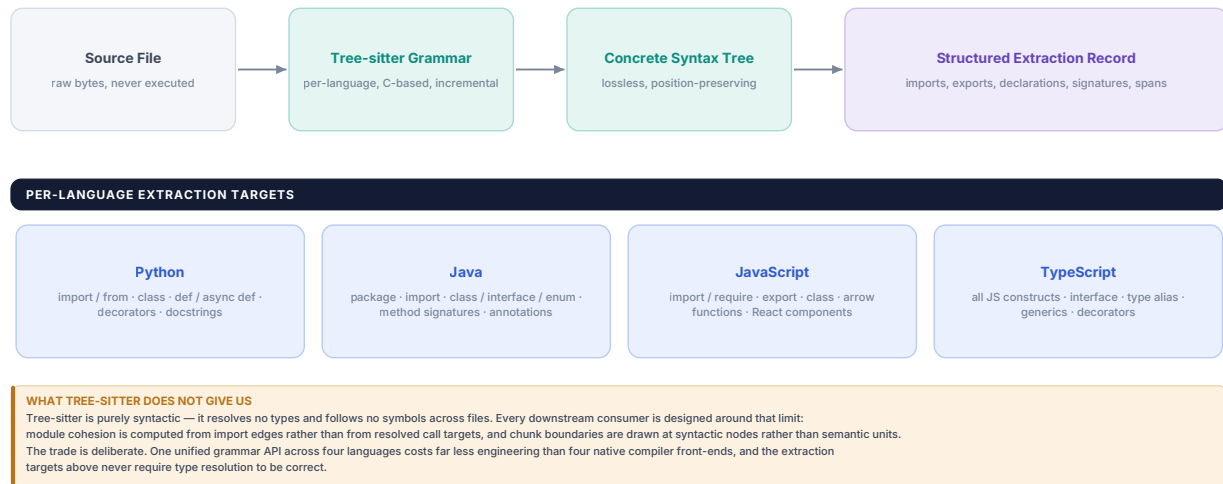


Figure 24.1 — Parsing path from raw bytes to a structured extraction record, with per-language targets. Tree-sitter builds syntax trees without compiling or executing source, which is what makes it compatible with the no-execution control in §18.2.

24.1 Extraction targets by language

Language	Extracted constructs
Python	<code>import</code> and <code>from ... import</code> statements; class declarations; <code>def</code> and <code>async def</code> definitions; decorator annotations; module and function docstrings
Java	Package declarations; import statements; class, interface and enum declarations; method signatures; Spring and Jakarta annotations
JavaScript	<code>import</code> and <code>require</code> statements; <code>export</code> statements; class declarations; arrow function assignments; React component signatures
TypeScript	All JavaScript constructs plus <code>interface</code> declarations, type aliases, generic parameters and decorators

24.2 What Tree-sitter does not provide

Tree-sitter is purely syntactic. It resolves no types, follows no symbols across files, and cannot determine that `db.query()` in one file refers to a method declared in another. Every downstream component is designed around this limit, and the design is better for having accepted it explicitly:

- Module cohesion is computed from **import edges** rather than resolved call targets (§25.2), because import statements are syntactically present and call resolution is not.
- Chunk boundaries are drawn at **syntactic nodes** rather than semantic units, which is sufficient because a function body is both.
- Framework detection uses **manifest files and import patterns** rather than type hierarchy analysis.

KNOWN CONSEQUENCE

Dynamic imports, runtime module resolution, and dependency injection wiring are invisible to this analysis. A repository that assembles its dependency graph at runtime will produce a sparser module graph than its actual coupling warrants. This is a real accuracy limit, it is reported in the evaluation error taxonomy (§41.2), and it is not silently hidden from the user.

25. Module Detection Algorithm

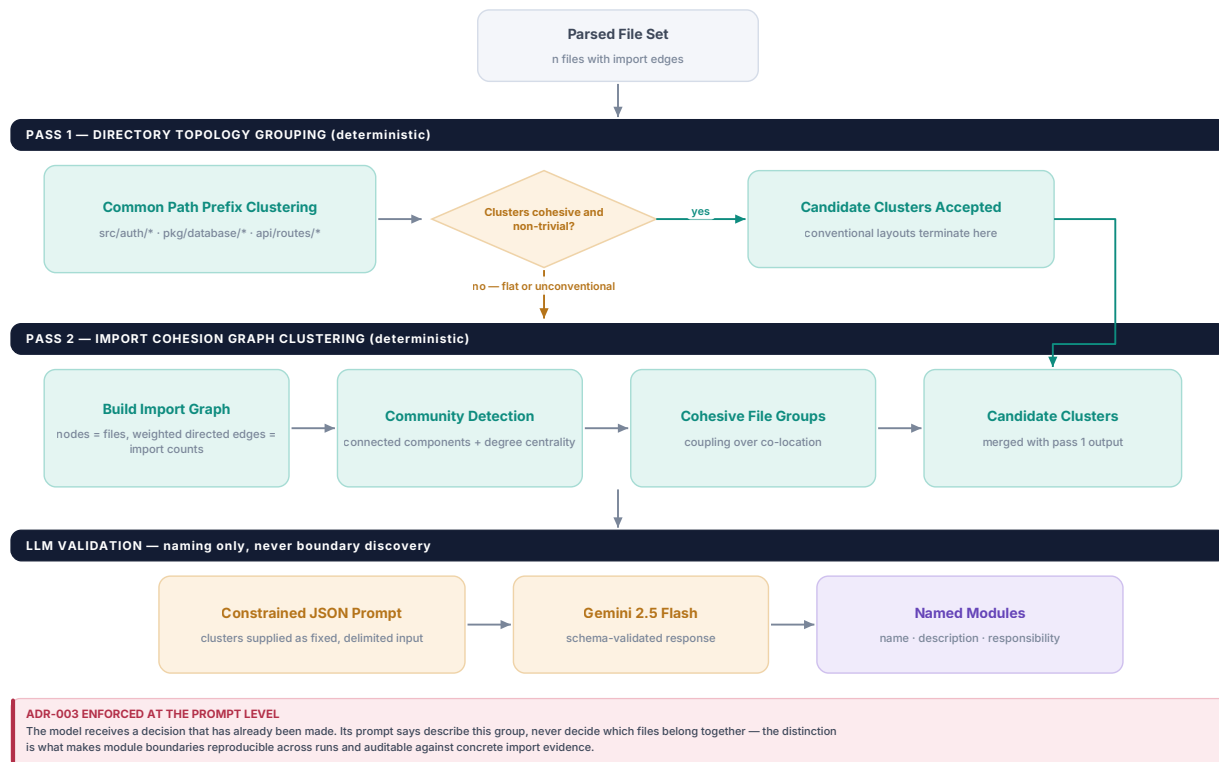


Figure 25.1 — Two deterministic passes, then naming. The model's involvement begins only after cluster membership is already fixed — the architectural enforcement of ADR-003.

25.1 Pass 1 — directory topology grouping

Files sharing a common structural path prefix — `src/auth/*`, `pkg/database/*`, `api/routes/*` — are grouped into initial clusters. For the majority of repositories built with mainstream framework scaffolding, directory structure already encodes logical module boundaries, and this pass alone is sufficient.

Pass 1 output is accepted when clusters are both **cohesive** (a cluster's files import one another more than they import outward) and **non-trivial** (the partition is neither one giant cluster nor one cluster per file). When either test fails — characteristic of flat layouts — pass 2 runs.

25.2 Pass 2 — import cohesion graph clustering

An import graph is constructed with files as nodes and weighted directed edges carrying import counts. Community detection — connected components combined with degree centrality — groups tightly coupled files.

The operating principle: files that import each other heavily belong to the same logical unit even when their directory placement does not say so. This is what allows the system to produce a meaningful map for a repository whose author never organised it into directories at all.

```
// Pass 2, in outline
G ← directed graph, V = parsed files, E = import edges weighted by count
components ← connected_components(G, undirected_view)

for each component in components:
    if size(component) > MAX_MODULE_SIZE:
        centrality ← degree_centrality(component)
        hubs      ← top-k nodes by centrality
        subgroups ← assign each node to its nearest hub by edge weight
        emit subgroups
    else:
        emit component

candidates ← merge(pass1_clusters, pass2_groups) // pass 1 wins ties
```

DEVELOPER NOTE — DETERMINISM REQUIREMENT

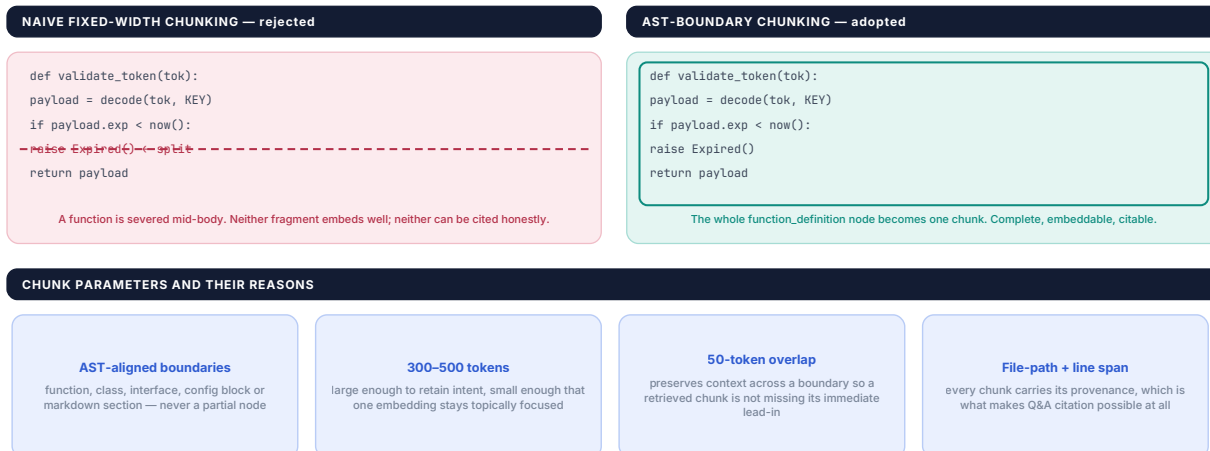
AC-07 requires byte-identical output across runs on identical input. This constrains implementation: iteration over graph nodes must occur in a sorted, stable order, and any tie-breaking in hub selection must be deterministic rather than dependent on set or dictionary ordering. This is the most common source of non-determinism in graph clustering code and must be tested for explicitly, not assumed.

25.3 LLM validation and naming

Candidate clusters are supplied to Gemini 2.5 Flash inside a constrained JSON schema prompt. The model returns, per cluster, a module name such as Authentication & Session Management, a concise description of what the cluster implements, and its operational responsibility within the application.

The prompt framing is deliberate and is the point at which ADR-003 becomes concrete: the instruction is describe this group, never decide which files belong together. Cluster membership is supplied as fixed input the model has no authority to alter, and the response schema contains no field through which it could.

26. Semantic Chunking



Chunking is where retrieval quality is actually decided. Every later component — embeddings, FAISS, RAG grounding, citation accuracy — inherits whatever this stage produces.

Figure 26.1 — Fixed-width versus AST-boundary chunking. The left case severs a function mid-body; neither fragment embeds meaningfully and neither can be cited honestly in an answer. Chunking is where retrieval quality is actually determined.

26.1 Parameters and their justification

Parameter	Value	Reasoning
Boundary rule	AST node	A chunk is always a complete function, class, interface, configuration block or markdown section. Partial nodes are never emitted.
Target size	300–500 tokens	Large enough to retain the intent of a unit; small enough that a single embedding stays topically focused. Larger chunks dilute the vector across multiple concerns and degrade retrieval precision.
Overlap	50 tokens	Preserves immediate lead-in context across boundaries, so a retrieved chunk is not missing the declaration or comment that gives it meaning.
Provenance	Path + line span	Every chunk carries its origin. Without this, citation in §29 is impossible — provenance must be captured at chunk time or it is lost forever.
Oversize handling	Split at nested nodes	A function exceeding the target is split at its next-level nodes rather than arbitrarily, preserving the boundary invariant.
Undersize handling	Merge siblings	Adjacent trivial declarations are merged to avoid an index full of near-empty vectors that match everything weakly.

WHY THIS STAGE DESERVES DISPROPORTIONATE CARE

Every later component inherits whatever chunking produces. Embedding quality, retrieval precision, grounding gate behaviour and citation accuracy are all downstream. A defect here is invisible in unit tests of later stages and manifests only as a diffuse degradation in answer quality — the hardest class of bug to localise. This is why §41 evaluates chunk-boundary integrity as a first-class metric rather than only measuring end-to-end answer quality.

27. Embeddings and Vector Indexing

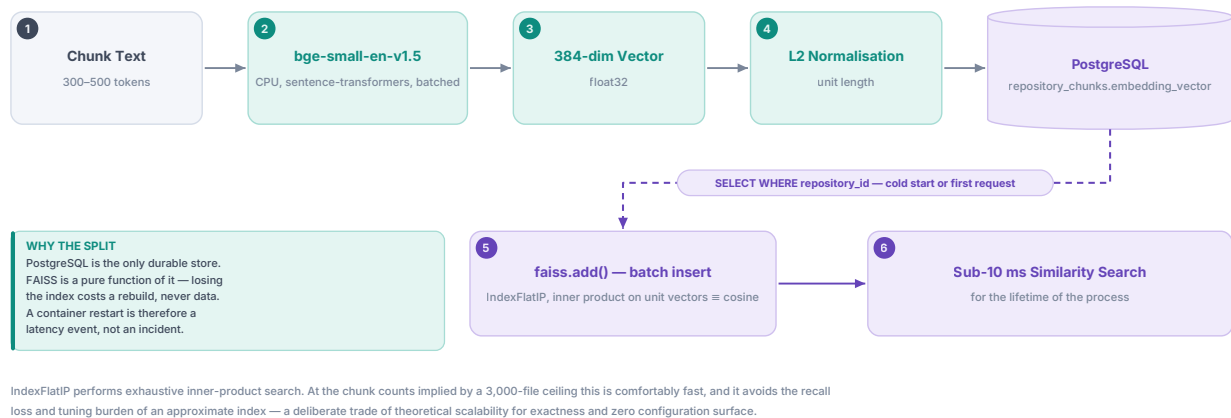


Figure 27.1 — Embedding generation and index reconstruction. PostgreSQL is the only durable store; FAISS is a pure function of it, which converts a potential data-loss incident into a bounded latency event.

27.1 Model selection

BAAI/bge-small-en-v1.5, 384 dimensions, run locally on CPU via `sentence-transformers`. The selection reasoning:

Candidate	Outcome	Reasoning
bge-small-en-v1.5	Selected	Strong retrieval performance per parameter on MTEB benchmarks; small enough for acceptable CPU batch throughput on commodity hardware; permissive licence; no API cost.
bge-large	Rejected	Higher quality, but memory footprint and CPU inference latency are incompatible with a free-tier container that also runs the API and parsers.
OpenAI text-embedding-3-small	Rejected	Excellent quality, but introduces a per-embedding API cost that violates the zero-budget constraint and a second external dependency in the critical path.
Code-specialised embeddings	Deferred	Plausibly better for this domain; deferred to \$49 as an open question because the evaluation harness needed to justify the swap does not exist until \$41 is built.

27.2 Normalisation and metric

Vectors are L2-normalised at both index time and query time, and searched with `faiss.IndexFlatIP`. On unit vectors, inner product is mathematically identical to cosine similarity — the metric the BGE family was trained and benchmarked under (ADR-008). `IndexFlatIP` performs exhaustive search, which at the chunk counts implied by a 3,000-file ceiling is comfortably fast and avoids both the recall loss and the tuning burden of an approximate index.

SILENT FAILURE WARNING

Omitting normalisation at either index time or query time does not raise an error. It produces plausible-looking but incorrectly-ranked results, and it will silently shift scores relative to the 0.60 gate — corrupting both retrieval quality and the hallucination control simultaneously. Normalisation is asserted in the test suite for exactly this reason.

27.3 Index reconstruction

On process start, or on first request against a repository not resident in memory, embeddings are read from `repository_chunks` filtered by `repository_id` and batch-inserted via a single `faiss.add()`. Batch insertion rather than per-vector addition matters: the per-call overhead dominates at the vector counts involved.

28. Hierarchical Map-Reduce Summarisation

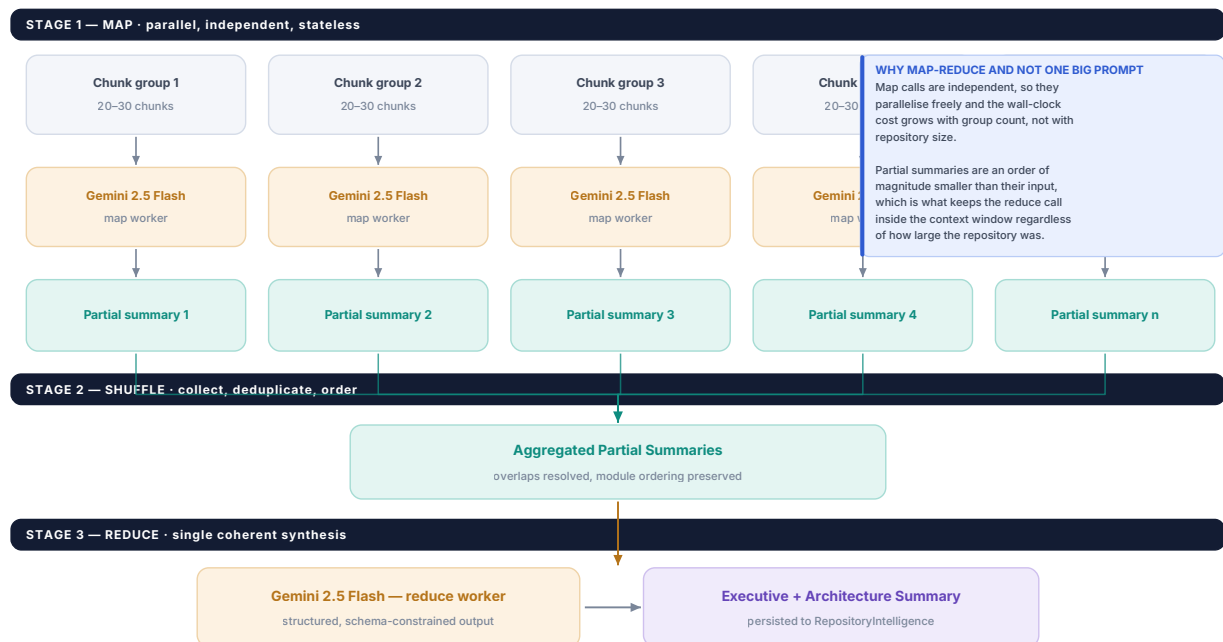


Figure 28.1 — Three-stage summarisation. Map calls are independent and parallelise freely; partial summaries are an order of magnitude smaller than their inputs, which is what keeps the reduce call inside the context window regardless of repository size.

28.2 Why not a single large prompt

Modern context windows are large enough that feeding an entire small repository to one call is technically possible. It was rejected for three reasons, only the first of which is about limits:

- **It does not scale past the envelope.** A 3,000-file repository exceeds any practical context window, so a single-prompt design would need a fallback path anyway — and maintaining two summarisation strategies is worse than maintaining one that always works.
- **Attention dilutes over long contexts.** Detail from the middle of a very long input is empirically less well represented in output than detail near either end. Hierarchical reduction gives every module a call in which it is the primary subject.
- **Failure granularity.** One failed map call costs one group and can be retried in isolation. One failed monolithic call costs the entire analysis.

28.3 Configuration

Parameter	Value	Note
Chunks per map task	20–30	Sized so a group fits comfortably in context with room for instructions and output
Concurrent map tasks	5–20	Bounded by the free-tier requests-per-minute quota rather than by our own capacity
Hierarchy depth	2 — map then reduce	Sufficient at the 3,000-file ceiling. The design admits an intermediate level if the ceiling rises
Map model	Gemini 2.5 Flash	Latency-optimised; the map phase is call-count-dominated
Reduce model	Gemini 2.5 Flash	Same model; input is already compressed, so a larger model would add latency without proportionate benefit
Output contract	JSON schema	Validated on every response; a schema failure is retried rather than silently rendered

29. Grounded RAG and Hallucination Control

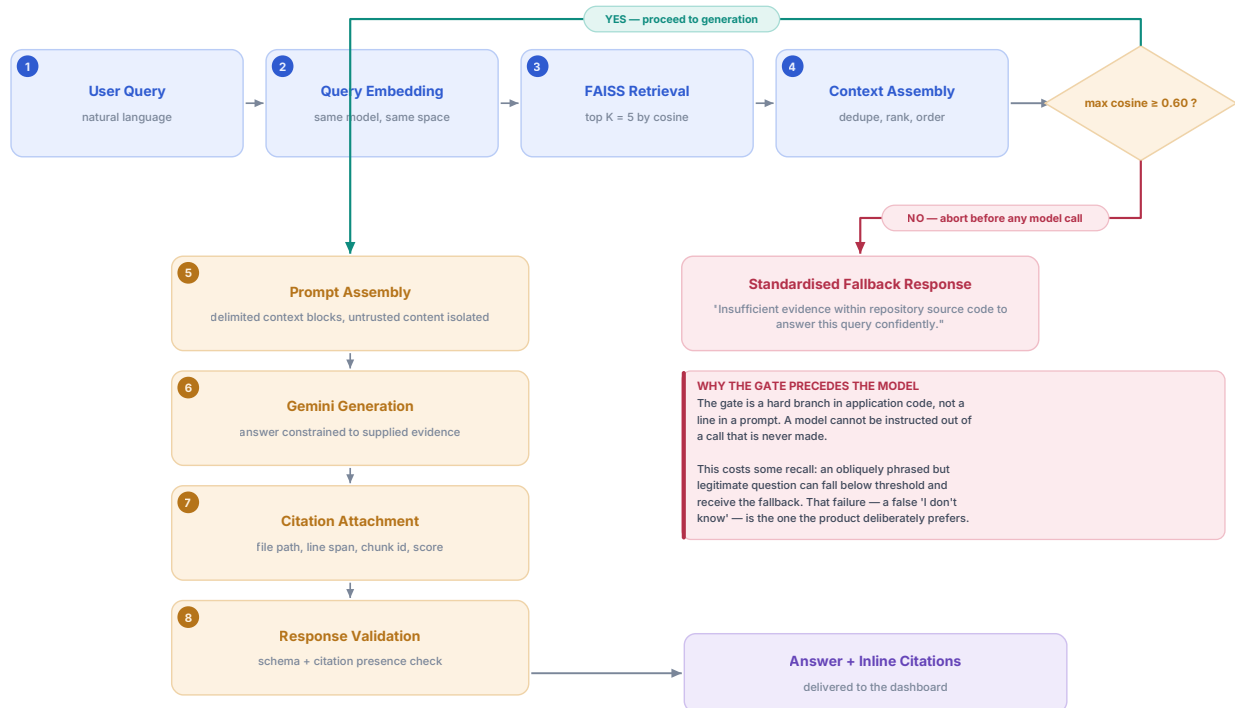


Figure 29.1 — The grounded answering path. The gate at step 4 is a branch in application code that precedes the model call — the "no" path never reaches generation at all, which is both a safety property and a cost property.

29.1 Retrieval sequence

Step	Detail
Query embedding	The query is embedded with the same model used at index time, guaranteeing both live in one vector space. Using a different model here is a subtle and catastrophic error.
Retrieval	Top K = 5 chunks by cosine similarity over the repository-scoped index.
Gate evaluation	Maximum similarity compared against the 0.60 floor. Below it, generation is skipped entirely.
Context assembly	Retrieved chunks deduplicated, ordered by relevance, and paired with their file paths and containing module metadata.
Generation	Structured prompt with delimited context; the model is instructed to answer only from supplied evidence and to cite for every substantive claim.
Citation attachment	Citations rendered as <code>[n] path (Lines X-Y)</code> with chunk identifier, module and score available for display.

29.2 Why the gate precedes the model

A PROMPT INSTRUCTION IS NOT A CONTROL

The common alternative — always generate, and instruct the model to say it does not know when unsure — is a weak mitigation. A model asked an out-of-scope question about a codebase can construct a fluent, entirely plausible answer from general programming knowledge without consulting the repository at all, and it has no reliable internal signal distinguishing that case from a grounded one. The retrieval score does have that signal. Placing the check where the signal exists, in application code, is the only version of this control that cannot be talked out of by an input.

29.3 The accepted cost

The gate produces false negatives: a legitimate question phrased obliquely can fall below threshold and receive the fallback. This is the deliberate direction of the trade. A false "I don't know" is recoverable — the user rephrases. A false confident answer is not recoverable, because the user has no signal that verification is needed. §41.2 measures the false-negative rate explicitly so the cost is quantified rather than assumed.

29.4 Stateless conversation handling

No chat history is persisted. The client sends the current query plus the last three turns with each request, supporting natural follow-ups within a session without the backend owning conversation state. This removes a table, a retention policy, a privacy consideration, and a scaling concern — in exchange for conversation context not surviving a page refresh, which ADR-007's sibling decision (Appendix C) records as acceptable for v1.0.

30. Repository Map Layout Engine

The map is the product's zero-input comprehension surface. Its construction is four steps, three deterministic and one generative.

Step 1 — Graph construction

A directed graph $G = (V, E)$ where V is the set of detected modules and E is the set of inter-module import couplings, aggregated from the file-level import edges established in Tier 1. Edge weight is the count of distinct file-level imports crossing the module boundary.

Step 2 — Layout calculation

G is passed to [@dagrejs/dagre](#), which computes 2D coordinates using a Sugiyama-style hierarchical layered algorithm. This algorithm was chosen because dependency graphs are hierarchical in practice — foundational modules such as data models have few dependencies and many dependents — and layered layout surfaces exactly that stratification. Force-directed layout, the common alternative, produces aesthetically balanced graphs that actively obscure hierarchy, which is the one property the reader most needs.

Step 3 — Frontend rendering

React Flow receives coordinates and edges and renders an interactive SVG canvas with pan, zoom and click-to-open module drawers. Layout is computed server-side and transmitted rather than computed in the browser, so the same graph is identical across clients and the layout cost is paid once per analysis rather than once per page load.

Step 4 — Narrative interpretation

The graph JSON is passed to Gemini 2.5 Flash to produce a prose overview of the system's structural flow, giving the reader both a visual and a narrative entry point into the same underlying fact. This is the only generative step in map construction, and it consumes the finished graph rather than influencing it.

31. Prompt Engineering Standards

Prompt templates are implementation artefacts, but the structural rules governing every model call are specification, because they operationalise DG-1 and the injection control in §18.4 at the point where they can actually fail.

TABLE 31.1 — MANDATORY PROMPT CONSTRUCTION RULES

#	Rule	Rationale
P1	Context and instruction MUST be structurally separated	Repository content is untrusted input. Placing it inside clearly delimited blocks, never concatenated into instruction position, is the primary defence against prompt injection.
P2	Output MUST be schema-constrained wherever it feeds a UI component	Schema validation converts a generation failure into a detectable, retryable error rather than malformed application state rendered to a user.
P3	Factual claims MUST require citation	Enforced by prompt instruction and by a post-generation validation step that rejects uncited substantive claims. Two independent mechanisms because one is a model behaviour and can fail.
P4	Structural decisions MUST be presented as fixed input	Module membership, file lists and dependency edges arrive as given facts. The response schema contains no field capable of altering them.
P5	Prompts MUST NOT request information absent from context	A prompt that asks about something not supplied invites fabrication. If the answer requires more evidence, the correct fix is retrieving more evidence, not asking harder.
P6	Every template MUST be versioned alongside code	A prompt change is a behaviour change. Untracked prompt edits make output regressions unattributable to any commit.

```
# Structural shape of every grounded call – rules P1, P2, P4

SYSTEM:
  You are a code intelligence assistant. Answer ONLY from the
  evidence in <context>. If the evidence does not support an
  answer, say so. Cite every claim as [n].

<context>                                # P1: delimited, never instruction position
  [1] auth/login.py (lines 45-86)
    <chunk content>
  [2] auth/session.py (lines 10-52)
    <chunk content>
</context>

<question>{user_query}</question>

RESPONSE SCHEMA:                        # P2: validated before render
{ "answer": str, "citations": [{"n": int, "path": str,
  "start_line": int, "end_line": int}] }
```

DEVELOPER NOTE — TESTING PROMPTS

Prompt behaviour is testable at two levels and both are required. Schema conformance is a deterministic assertion and belongs in the unit suite. Grounding behaviour — does the model actually decline when context is insufficient — requires a fixture set of known-unanswerable questions and belongs in the evaluation harness in §41, where a regression in refusal behaviour is measurable rather than anecdotal.

PART FIVE

05

Implementation Reference

Everything required to build the system without further clarification: the stack with its rejected alternatives, the schema, the API contracts, authentication, concurrency, caching and the frontend.

-
- 32 Technology Stack
 - 33 Database Design and Data Dictionary
 - 34 REST API Specification
 - 35 Authentication and Authorization
 - 36 Background Processing and Concurrency
 - 37 Caching Strategy
 - 38 Report Compilation Engine
 - 39 Frontend Architecture

32. Technology Stack and Rejected Alternatives

TABLE 32.1 — EVERY SELECTION WITH ITS REJECTED ALTERNATIVE

Layer	Selected	Justification	Alternative	Why not
Frontend framework	Next.js 15 / React 19	Server rendering for fast first paint, App Router for layout composition, native TypeScript	Vite + SPA	Would require hand-building routing, SSR and asset optimisation for no capability gain
UI primitives	Tailwind + shadcn/ui	Minimal, accessible, unopinionated primitives supporting rapid consistent construction	Material UI	Heavy visual defaults conflict with the restrained design language in §11.7
Graph rendering	React Flow + Dagre	Node-graph rendering with pan, zoom, custom nodes; automated layered layout	Raw D3	Substantially more implementation for equivalent interactivity
Backend framework	FastAPI, Python 3.12	Async-native, Pydantic validation, automatic OpenAPI generation	Flask, Django REST	Neither offers native async plus schema-derived documentation, both of which reduce API-layer effort materially
Parser	Tree-sitter	Unified API across four languages, no compilation, no execution	Native parsers	ADR-006 — four code paths quadruple maintenance surface
Embedding model	bge-small-en-v1.5	Strong retrieval per parameter, CPU-viable, permissive licence, no API cost	text-embedding-3	Per-call cost violates the budget constraint; adds a critical-path dependency
Vector index	FAISS (faiss-cpu)	Optimised C++ similarity search, in-memory, zero configuration	Managed vector DB	ADR-002 — free-tier ceilings and a third external dependency
Generative model	Gemini 2.5 Flash	Large context, fast inference, workable free tier	Self-hosted open weights	ADR-005 — free-tier compute cannot host it at acceptable latency
Database	Neon PostgreSQL	Serverless managed Postgres; array and JSONB types hold vectors beside relational data	Supabase / PlanetScale + separate vector store	Reintroduces the two-store consistency problem ADR-002 exists to avoid
ORM	SQLAlchemy	Mature, explicit, migration tooling via Alembic	Raw SQL	Loses migration management and type safety across a schema this size
PDF engine	WeasyPrint	Python-native HTML/CSS to PDF; same template stack as the rest of the backend	Headless Chromium	Adds a Node runtime and a browser binary inside a Python container for no gain
Task execution	FastAPI BackgroundTasks	Zero additional infrastructure; adequate at target concurrency	Celery + Redis	Premature — added when a bottleneck is measured (§19.2), not before

33. Database Design and Data Dictionary

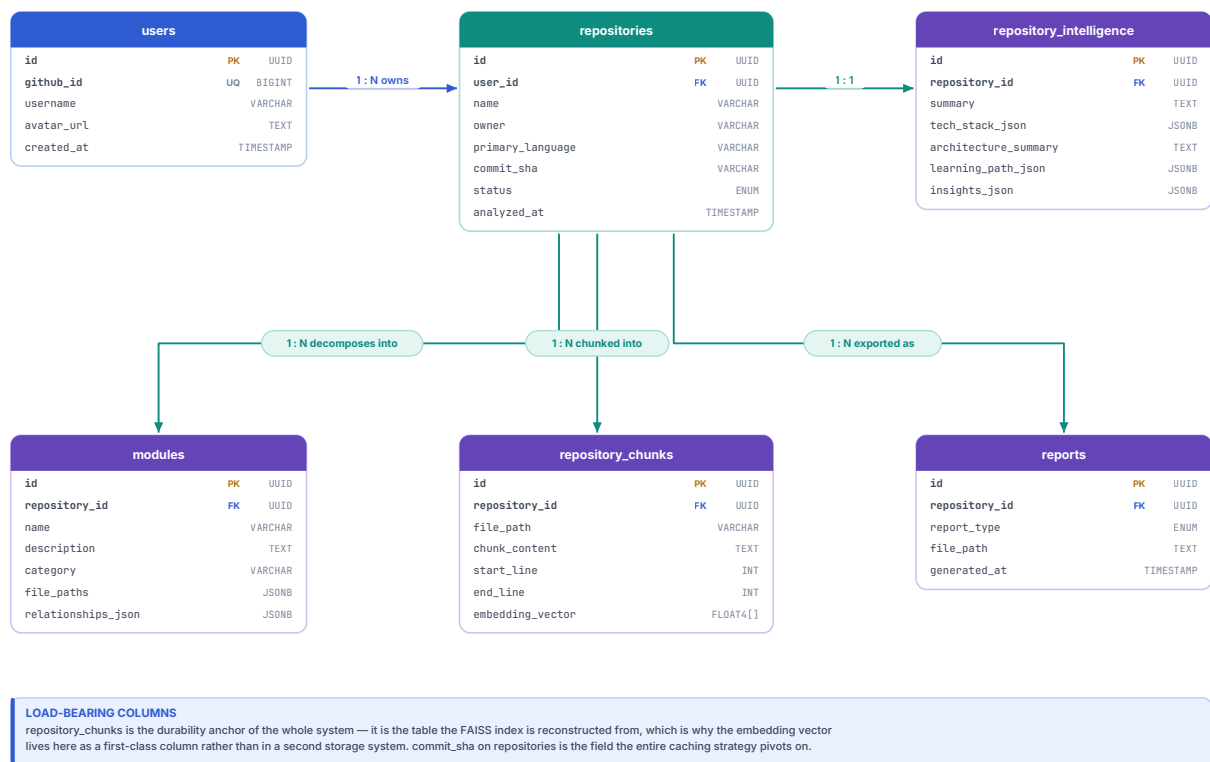


Figure 33.1 — Entity relationships. A user owns many repositories; each repository yields exactly one intelligence model but decomposes into many modules, chunks and reports. `repository_chunks` is the durability anchor of the entire system.

33.1 Data dictionary

users

Column	Type	Key	Semantics
<code>id</code>	UUID	PK	Internal identifier; never exposed to GitHub
<code>github_id</code>	BIGINT	UNIQUE	GitHub's numeric user identifier — the stable join key, since usernames can be changed by their owner
<code>username</code>	VARCHAR(255)		Display only. Explicitly not used as an identity key
<code>avatar_url</code>	TEXT		Display only
<code>created_at</code>	TIMESTAMP		First authentication timestamp

repositories

Column	Type	Key	Semantics
<code>id</code>	UUID	PK	Analysis subject identifier
<code>user_id</code>	UUID	FK	Owning user; scopes every query at the data-access layer
<code>name</code> , <code>owner</code>	VARCHAR(255)		GitHub coordinates; unique together per user
<code>primary_language</code>	VARCHAR(50)		Dominant language by file count, computed in Tier 1
<code>commit_sha</code>	VARCHAR(64)	INDEX	Cache key. The analysed commit. Every caching decision in §37 reads this column
<code>status</code>	ENUM	INDEX	QUEUED, PROCESSING, COMPLETED, FAILED — the value the polling endpoint returns
<code>analyzed_at</code>	TIMESTAMP		Completion time of the most recent successful analysis

repository_chunks

Column	Type	Key	Semantics
<code>id</code>	UUID	PK	Chunk identifier, surfaced in citations
<code>repository_id</code>	UUID	FK, INDEX	Scoping key for index reconstruction — this index is performance-critical
<code>file_path</code>	VARCHAR(1024)		Provenance. Required for citation rendering
<code>chunk_content</code>	TEXT		The chunk text supplied to the model as grounded context
<code>start_line</code> , <code>end_line</code>	INT		Line span. Without these, a citation cannot point at anything verifiable
<code>embedding_vector</code>	FLOAT4[]		384 L2-normalised floats. The durable source from which FAISS is rebuilt

INDEX STRATEGY

Three indices carry the system: `repositories(commit_sha)` backing the cache check on every selection, `repository_chunks(repository_id)` backing index reconstruction, and `users(github_id)` backing the auth upsert. Everything else is served by primary keys. Additional indices would add write cost during the bulk chunk insert — the single heaviest write in the pipeline — for queries that do not occur.

33.2 Remaining entities

Entity	Purpose and notable columns
<code>repository_intelligence</code>	One row per repository. Holds <code>summary</code> , <code>tech_stack_json</code> , <code>architecture_summary</code> , <code>learning_path_json</code> , <code>insights_json</code> . This is the aggregate the dashboard and both export formats read — the single contract three consumers share, which is why format parity (AC-15) is structural rather than maintained by discipline.
<code>modules</code>	One row per detected module. <code>file_paths</code> holds membership as JSONB; <code>relationships_json</code> holds the outbound edges backing the map. <code>category</code> drives node colouring.
<code>reports</code>	Tracks generated export artefacts so a repeat download does not re-render. <code>report_type</code> is PDF or MARKDOWN.

34. REST API Specification

TABLE 34.1 — ENDPOINT SURFACE

Method	Path	Auth	Behaviour
GET	<code>/api/v1/auth/github</code>	Public	Issues the OAuth redirect with read-only scopes and a state parameter
GET	<code>/api/v1/auth/callback</code>	Public	Validates state, exchanges code for token, upserts the user, returns a signed JWT
GET	<code>/api/v1/repositories</code>	Bearer	Lists accessible public repositories with metadata for the selection view
POST	<code>/api/v1/analysis/trigger</code>	Bearer	Resolves remote SHA, compares to stored, returns 200 with cached reference or 202 with an analysis identifier
GET	<code>/api/v1/analysis/{id}/status</code>	Bearer	Returns the current stage and progress; polled at 2 s
GET	<code>/api/v1/intelligence/{repo_id}</code>	Bearer	Returns the complete intelligence model
GET	<code>/api/v1/map/{repo_id}</code>	Bearer	Returns nodes with Dagre coordinates plus edges, ready for React Flow
POST	<code>/api/v1/search/semantic</code>	Bearer	Embeds the query and returns ranked chunks with scores and module context
POST	<code>/api/v1/ask</code>	Bearer	Grounded answering with the similarity gate; accepts query plus last three turns
POST	<code>/api/v1/report/export</code>	Bearer	Renders and returns a PDF or Markdown artefact

34.1 Contract detail for the two non-obvious endpoints

POST `/api/v1/analysis/trigger`

```
// Request
{ "owner": "vercel", "name": "next.js" }

// 200 - cache hit, no work scheduled
{ "cached": true, "repository_id": "3f1c...", "commit_sha": "a94b...",
  "analyzed_at": "2026-03-11T09:22:14Z" }

// 202 - analysis scheduled
{ "cached": false, "analysis_id": "b72e...", "stage": "QUEUED" }

// 413 - outside the operating envelope
{ "error": "repository_too_large", "file_count": 8412, "limit": 3000 }
```

POST /api/v1/ask

```
// Request
{ "repository_id": "3f1c...",
  "query": "How is the session token validated?",
  "history": [ {"role":"user","content":"..."},
               {"role":"assistant","content":"..."} ] }

// 200 – gate cleared
{ "grounded": true,
  "answer": "Session tokens are validated in ...[1]... before ...[2].",
  "citations": [
    {"n":1,"path":"auth/session.py","start_line":10,"end_line":52,
     "module":"Authentication","score":0.8713},
    {"n":2,"path":"auth/verify.py","start_line":15,"end_line":62,
     "module":"Authentication","score":0.8047} ] }

// 200 – gate not cleared; note grounded:false and empty citations
{ "grounded": false, "max_score": 0.41, "citations": [],
  "answer": "Insufficient evidence within repository source code
            to answer this query confidently." }
```

WHY THE FALLBACK RETURNS 200

A declined answer is a successful, correct response — the system did exactly what it should. Returning 4xx would signal client error and would cause well-behaved clients to retry, which would be both wasteful and wrong. The `grounded` boolean carries the distinction, and the client renders the two cases differently.

34.2 Cross-cutting conventions

- **Versioning by path prefix.** Breaking changes require `/api/v2`; in-place breaking changes to `v1` are prohibited (§47).
- **Errors carry a machine-readable code** plus a human-safe message. Never a stack trace (§11.7).
- **All list responses are envelope-wrapped** so pagination can be added later without a breaking change.
- **Every authenticated endpoint scopes by the JWT subject** at the data-access layer, not in the route handler — the check cannot be forgotten in a new route because it lives below them all.

35. Authentication and Authorization

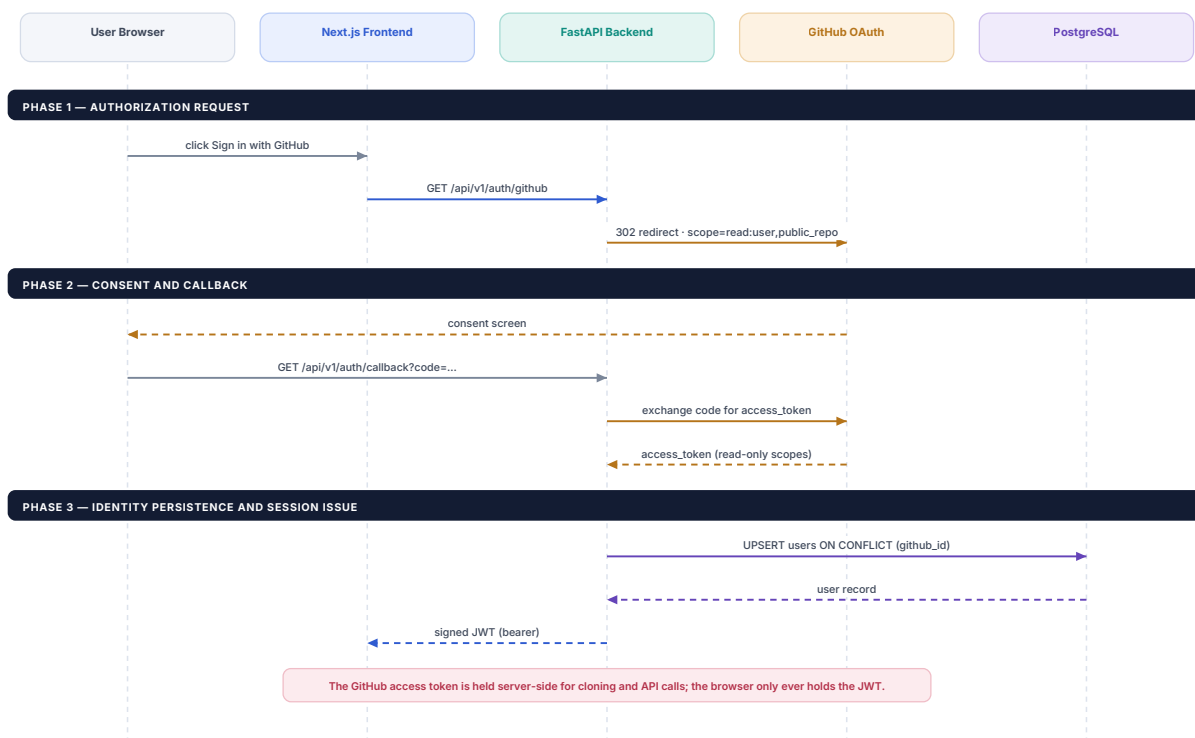


Figure 35.1 — OAuth flow across three phases. The GitHub access token is retained server-side for cloning and API calls; the browser holds only the signed JWT and never sees the upstream credential.

35.1 Session model

On successful callback the backend issues a short-lived signed JWT, attached as a bearer token on subsequent requests. Server-side session storage was rejected (Appendix C, ADR-009): it adds a stateful service dependency for a model that needs no server-side revocation, since no destructive capability exists behind the session to revoke access to. Logout is client-side token discard.

35.2 Authorization model

Authorization is single-tier. Every authenticated user has identical rights over their own analysed repositories, and there is no shared, organisational or administrative surface. Role-based access control would add schema, middleware and test surface with no corresponding requirement — a deliberate application of the scope discipline in §6.2 rather than an omission.

36. Background Processing and Concurrency

```
# The workspace contract — every analysis job is shaped exactly like this

with tempfile.TemporaryDirectory(prefix="repolens_") as tmp_dir:
    # 1. authenticated shallow clone into tmp_dir
    # 2. Tree-sitter parse and metadata extraction
    # 3. two-pass module detection, then LLM naming
    # 4. chunking, embedding, vector persistence to PostgreSQL
    # 5. Map-Reduce intelligence generation and final commit
# tmp_dir is destroyed here on every exit path, including exceptions.
# This is why NFR-04 is a guarantee rather than a convention.
```

36.1 Concurrency model

`BackgroundTasks` executes in the same process as the request event loop. I/O-bound stages — GitHub calls, Gemini calls, database writes — use `async` / `await` so a running analysis does not block status polling for other jobs or dashboard reads for completed ones.

THE HONEST LIMITATION

Tree-sitter parsing and embedding inference are synchronous and CPU-bound. `async` provides no concurrency for them. A large analysis will occupy a worker and reduce throughput for co-resident requests. This is accepted at target concurrency, stated explicitly rather than discovered in production, and the remedy — a worker process pool or an external queue — is recorded in §46 with the condition that would trigger it.

37. Caching Strategy

Two caching mechanisms exist. Both address a measured cost centre; no third was added.

Primary — commit-SHA-gated intelligence cache

Detailed in §11.2 and Figure 11.2. A repository whose remote HEAD matches the stored analysis SHA bypasses the entire pipeline. This is the dominant lever because it eliminates the two most expensive operations — embedding compute and generative API calls — simultaneously and completely.

Secondary — in-process FAISS residency

Once a repository's vectors are resident in a process, subsequent search and answering requests avoid reconstruction. Effectively a read-through cache over PostgreSQL with process lifetime as its eviction policy.

WHY NO THIRD LAYER

An HTTP response cache backed by Redis was considered and rejected. The two mechanisms above already address both dominant cost centres; a third would add an external service, an invalidation surface, and a class of staleness bug, in exchange for shaving milliseconds off responses that are already inside their NFR budget. Caching layers are not free, and adding one that solves nothing is a net regression.

38. Report Compilation Engine

The engine reads the persisted `repository_intelligence` row and renders it through one of two Jinja2 template sets. The PDF path applies CSS print media rules — `@page` sizing, running headers, page numbering — and compiles through WeasyPrint. The Markdown path emits GitHub-Flavored Markdown from a parallel template.

STRUCTURAL PARITY

Both formats and the live dashboard consume the identical intelligence model. Content parity between them (AC-15) is therefore a structural property, not a maintenance obligation — there is no second rendering path holding its own copy of the data that could silently drift. Adding a third export format would mean adding a template, not adding a synchronisation problem.

Report contents: title and metadata block, metrics table, module breakdown with responsibilities, dependency observations, architectural insights, and the ordered learning path. Generated artefacts are recorded in `reports` so a repeat download of an unchanged analysis does not re-render.

39. Frontend Architecture

The Next.js 15 application uses the App Router, with a clean split between the public landing surface and the authenticated dashboard surface. The split is structural — separate route groups with separate layouts — so that no authentication-dependent code can be reached from a public route by accident.

39.1 State management

Dashboard state — the loaded intelligence model, analysis progress, search and conversation state — is held in React state and context rather than an external state library. The application has one primary data object per route and no cross-route state synchronisation problem, which is precisely the condition under which a state management library adds ceremony without solving anything.

39.2 The map subtree

React Flow is isolated as a dedicated component subtree with its own memoisation boundary. Its canvas rendering model and internal state are unlike the rest of the dashboard, and isolating it prevents unrelated dashboard re-renders from triggering expensive graph re-layout — the single most common performance defect in React Flow integrations.

39.3 Polling and progress

The progress view polls the status endpoint on a two-second interval with cleanup on unmount, and stops on terminal states. A stall threshold surfaces a retry affordance rather than polling indefinitely against a job whose worker has died (§21.1).

PART SIX

06

Quality, Evaluation and Execution

How correctness is verified, how quality is measured against a fixed benchmark suite, and how the work is divided, sequenced and de-risked across a twelve-week delivery.

-
- 40 Testing Strategy
 - 41 Evaluation Framework
 - 42 Performance Benchmarks
 - 43 Team Ownership
 - 44 Implementation Roadmap
 - 45 Risk Register

40. Testing Strategy

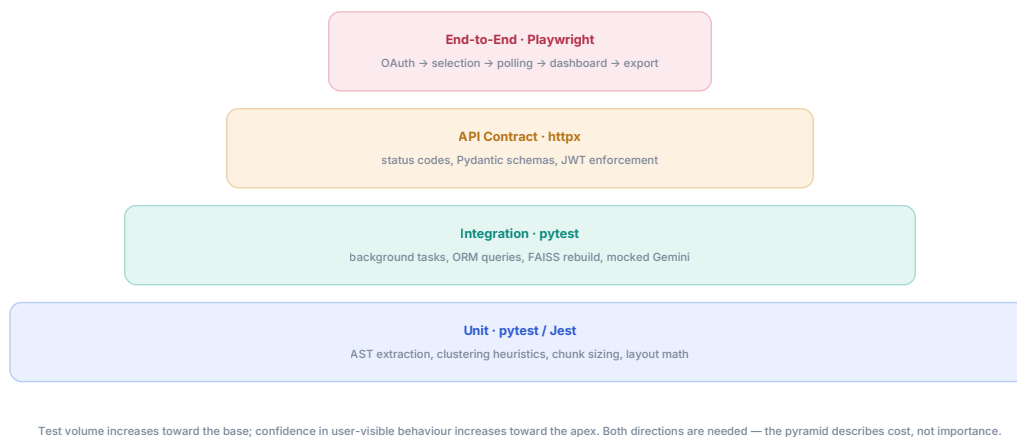


Figure 40.1 — Four test layers. Volume increases toward the base and confidence in user-visible behaviour increases toward the apex; the shape describes relative cost, not relative importance.

TABLE 40.1 — TEST MATRIX

Layer	Tooling	Coverage	Highest-value assertions
Unit	pytest, Jest	AST extraction per language, clustering heuristics, chunk sizing and boundary integrity, embedding shape and normalisation, Dagre output validity	Clustering determinism (AC-07); vector normalisation, whose absence fails silently (§27.2)
Integration	pytest + test database	Background task execution, ORM queries and migrations, FAISS reconstruction from persisted vectors, Gemini client with mocked transport	Retrieval parity between a warm index and one rebuilt from a cold process
API contract	httpx, Postman	Status codes, Pydantic request and response schemas, JWT enforcement on every authenticated route	Negative auth tests; the fallback path returning 200 with <code>grounded:false</code> (§34.1)
End-to-end	Playwright	OAuth login, repository selection, progress polling, map interaction, search, question answering, export download	Cache-hit path enqueues zero jobs (AC-03); export artefact opens and is non-empty

40.1 CI gate

GitHub Actions runs backend and frontend suites on every pull request. Both must pass before merge; deployment is triggered by merge to `main` through Vercel's and Render's native Git integrations (§17.2). CI gates quality; platform integrations handle delivery.

DEVELOPER NOTE — TESTS THAT EARN THEIR KEEP

Three assertions in this matrix have caught the most defects in systems of this shape and should be written first: clustering determinism, vector normalisation, and gate behaviour below threshold. Each guards a failure that is silent — producing plausible wrong output rather than an error — which is exactly the category that survives casual manual testing and reaches users.

41. Evaluation Framework

41.1 Benchmark suite

Ten diverse public repositories spanning all four supported languages, selected to include both conventionally organised and deliberately flat layouts so that both detection passes (§25) are exercised. The same suite is used for every benchmark run so that results remain comparable across the development period — a suite that changes cannot detect a regression.

TABLE 41.1 — EVALUATION DIMENSIONS

Dimension	Method	Pass condition
Functional correctness	Full pipeline across all ten repositories	Every repository completes; map renders; no stage fails systematically for a language
Module boundary quality	Manual review against reviewer-assigned boundaries	Majority agreement on major modules; disagreements categorised, not just counted
Groundedness	Fixture set of answerable and unanswerable questions	Zero fabricated answers below the 0.60 gate — a hard pass condition, not a rate
Citation accuracy	Sampled answers verified against cited spans	Cited line ranges genuinely support the claim they are attached to
Chunk integrity	Automated boundary assertions	No chunk begins or ends mid-node
Performance	Timing against NFR-01 to NFR-03	Targets met per size class (§42)

41.2 Error taxonomy

Failures are categorised rather than merely counted, because an aggregate error rate does not tell an engineer what to fix. The categories tracked:

- **Retrieval miss** — evidence existed in the repository but ranked below K. Points at chunking or embedding.
- **Gate false negative** — legitimate question declined. The measured cost of the trade in §29.3.
- **Boundary disagreement** — clustering produced a defensible-but-different partition than a human reviewer. Often not a defect.
- **Dynamic-structure blindness** — coupling invisible to static analysis (§24.2). A known and reported limit.
- **Citation drift** — the cited span does not support the claim. The most serious category; treated as a correctness bug, not a quality issue.

42. Performance Benchmarks

TABLE 42.1 — TARGET ENVELOPE BY REPOSITORY SIZE CLASS

Class	Files	Approx. chunks	Analysis target	Role in the suite
Small	500–1,000	2,000–4,000	Comfortably inside NFR-01	Baseline correctness; fastest feedback during development
Medium	1,000–2,000	4,000–8,000	Approaching NFR-01	The representative case; NFR-01 is defined against this class
Large	2,000–3,000	8,000–12,000	At the envelope edge	Stress case; validates Map-Reduce bounding and index memory

Targets are deliberately conservative relative to raw stage costs (§19.1), leaving headroom for external API latency variance the system does not control. A benchmark that only passes on a good day for the upstream provider is not a benchmark.

43. Team Ownership

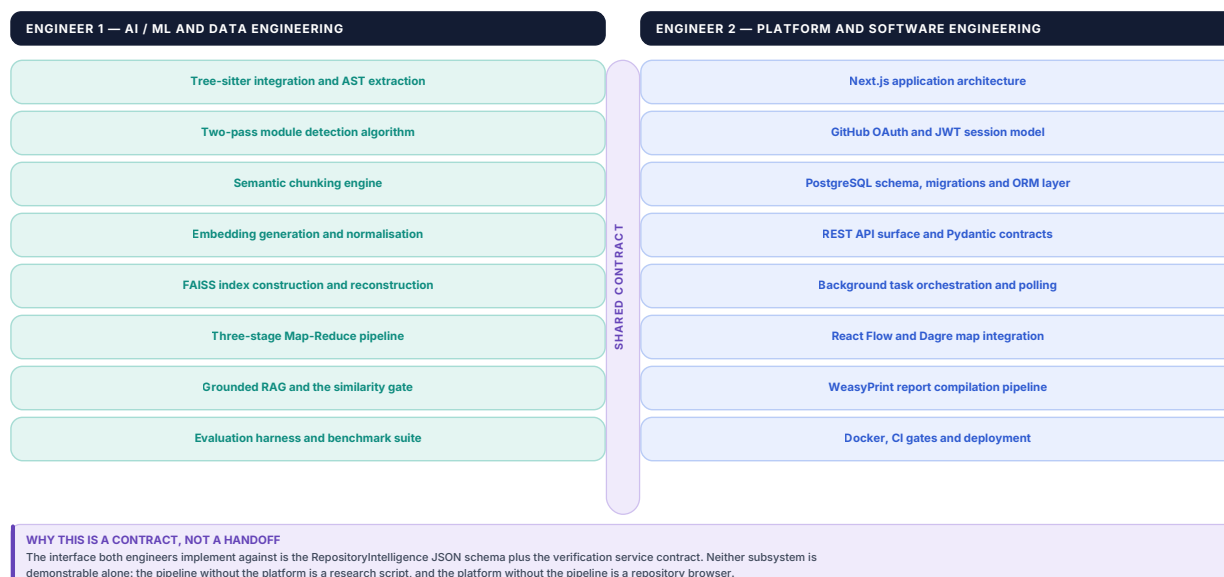


Figure 43.1 — Ownership split with the shared contract between them. Neither column is demonstrable alone — the pipeline without the platform is a research script; the platform without the pipeline is a repository browser.

43.1 Why this is a contract, not a handoff

The interface both engineers implement against is the intelligence model schema (§33) plus the internal service contract between the engines. This is a genuine two-way dependency: the platform must accommodate the pipeline's variable latency and probabilistic output, while the pipeline must emit structures the map, citations and export templates can consume directly. Neither side can specify it alone, which is what makes the collaboration substantive rather than sequential.

44. Implementation Roadmap

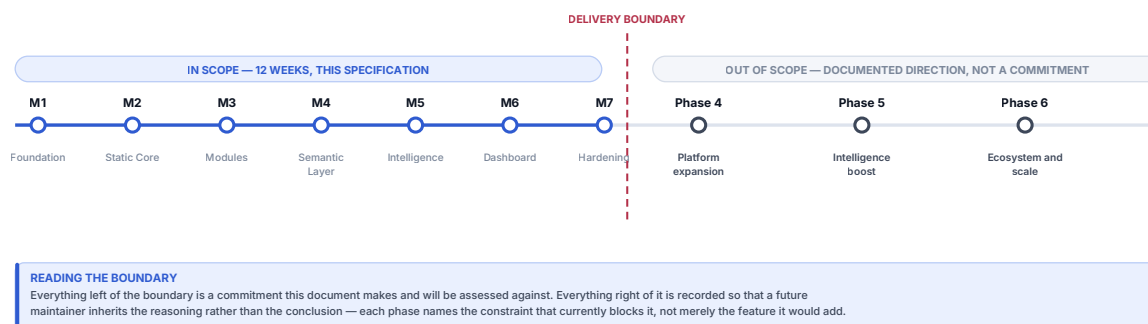


Figure 44.1 — Seven milestones inside the delivery boundary, and three phases beyond it. Everything left of the boundary is a commitment; everything right is recorded direction.

TABLE 44.1 — MILESTONE DEFINITION

ID	Focus	Deliverables	Exit criterion
M1	Foundation	OAuth end to end, Next.js and FastAPI skeletons deployed, initial schema migrated	A user can log in and see an empty dashboard on production URLs
M2	Static core	Cloning, four-language parsing, import extraction, workspace lifecycle	A repository produces a correct structural record; NFR-04 asserted
M3	Module detection	Both clustering passes plus LLM naming	Deterministic output verified across repeated runs (AC-07)
M4	Semantic layer	Chunking, embedding, persistence, FAISS reconstruction	Search returns relevant results; rebuilt index matches warm index
M5	Intelligence	Map-Reduce summarisation, grounded Q&A, graph layout	Gate verified to make zero model calls below threshold (AC-14)
M6	Dashboard and reports	All six capabilities, both export formats, learning path	Full journey completable in the browser; export parity asserted
M7	Hardening	Test suite green, benchmark suite executed, edge cases covered	All NFRs measured and reported; evaluation documented

45. Risk Register

TABLE 45.1 — RISKS WITH LIKELIHOOD, IMPACT AND MITIGATION

Risk	Likelihood	Impact	Mitigation and residual exposure
GitHub rate limiting	Medium	High	Per-user OAuth token elevation to 5,000 req/hr (§11.1). Residual: a single user running many analyses concurrently could still exhaust their own budget.
Ephemeral storage loss	Medium	High	PostgreSQL as system of record; FAISS rebuilt on demand (ADR-002). Residual: bounded cold-start latency, explicitly excluded from NFRs.
LLM context or latency limits	Medium	Medium	Map-Reduce bounds context at every stage (§28). Residual: very large module counts increase call volume against RPM quota.
Gemini quota exhaustion	Medium	Medium	SHA caching avoids redundant generation; token accounting predicts exhaustion (§20.1). Residual: generative capabilities degrade — but per §21.2 the system remains partially functional.
Polyglot or unconventional structures	Medium	Medium	Two-pass detection handles flat layouts (§25.2); coverage labelled honestly when partial (§12.2).
Team bandwidth over 12 weeks	Medium	High	Independently progressable ownership domains (§43) allow parallel rather than sequential work; milestone exit criteria surface slippage early rather than at the end.
Prompt injection via repository content	Low	Medium	Delimited context blocks (§31, P1) plus gate-bounded blast radius. Residual: a crafted repository could degrade its own Q&A quality, affecting no other user.
Disk exhaustion from clone residue	Low	Low	Context-manager-guaranteed deletion asserted on exception paths (NFR-04).

PART SEVEN

07 Forward Direction and Appendices

What comes after v1.0 and what remains unanswered — plus the reference material required to build, operate and extend the system: glossary, decision index, API card, schema and citations.

46 Future Roadmap

47 Versioning and API Evolution

48 Multi-Language Expansion Path

49 Open Research Questions

A–F Appendices, Glossary and References

46. Future Roadmap

Each item below names the constraint currently blocking it rather than only the feature it would add, so that a future maintainer inherits the reasoning and can recognise when the constraint has lifted.

TABLE 46.1 — DEFERRED CAPABILITIES AND THEIR BLOCKING CONSTRAINTS

Capability	Blocking constraint today	Unblocking condition
Additional language parsers — C/C++, Rust, Go, Ruby	Engineering time per grammar plus language-specific extraction rules; not a design limit	Available engineering capacity. The pipeline downstream of parsing is already language-agnostic (§48)
Private repository support	Requires higher-trust OAuth scopes and correspondingly hardened credential handling beyond §18	A security review of token storage at rest and a documented incident response path
Multi-repository comparison	Data model and UI complexity; no established demand at v1.0	Evidence that users analyse related repositories in sequence — measurable from usage before building
Dedicated task queue — Celery and Redis	Premature. In-process tasks are adequate at current concurrency	Measured queueing delay or CPU contention between concurrent analyses (§19.2)
Persistent conversation history	Adds a table, a retention policy and a privacy consideration for value not yet demonstrated	User evidence that context loss across sessions is a genuine friction rather than a theoretical one
IDE extension	A second client surface to build and maintain against an API still stabilising	A stable v1 API contract and demonstrated web-surface adoption
Self-hosted observability — Grafana and Loki	Operationally disproportionate for a single instance read by two engineers	Multiple instances, or a debugging incident the platform log viewer could not resolve
Approximate vector index — HNSW or IVF	Exhaustive search is fast enough inside the current envelope; approximation would add recall loss and tuning burden for no gain	A raised file-count ceiling that makes exhaustive search exceed NFR-02

THE DISCIPLINE THIS TABLE ENCODES

Every row states a condition that can be observed, not a date. This is deliberate: a roadmap of dates decays into a backlog of guilt, whereas a roadmap of triggers remains accurate indefinitely and tells a future maintainer what to measure. The recurring theme — build infrastructure when a bottleneck is measured, not when it is imagined — is the same principle applied in §19.2, §20.2 and §37.

47. Versioning and API Evolution

The REST surface is versioned by URL path prefix. The governing rule: any change that would break a conforming `v1` client requires `/api/v2`, never an in-place modification.

TABLE 47.1 — CHANGE CLASSIFICATION

Change type	Treatment	Examples
Additive field in a response	Non-breaking	New metadata on a module node. Clients ignoring unknown fields are unaffected
New optional request parameter	Non-breaking	An optional filter on semantic search
New endpoint	Non-breaking	Any capability added within <code>v1</code>
Field removal or rename	Breaking	Requires <code>v2</code>
Type or semantics change	Breaking	Score scale change; enum value redefinition
Required parameter added	Breaking	Requires <code>v2</code>

Header-based and content-negotiated versioning were considered and rejected: both solve the problem of multiple independent consumers needing different versions simultaneously, which does not exist for a system with one first-party client. Path versioning is legible in logs, trivially routable, and obvious in a browser address bar — properties that matter more here than the elegance of the alternatives.

48. Multi-Language Expansion Path

Adding a language is a bounded, four-step recipe rather than a redesign. This is a direct dividend of the Tree-sitter decision in ADR-006 and the language-agnostic design of everything downstream of parsing.

1. **Integrate the Tree-sitter grammar.** Add the grammar package and register it against the file extensions it claims.
2. **Define extraction rules.** Specify the node types corresponding to imports, exports, declarations and signatures — the equivalent of one row in the §24.1 table.
3. **Validate the detection heuristics.** Confirm that directory topology and import cohesion behave sensibly against representative repositories. Adjust heuristic parameters only if a language idiom demands it — Go's package-per-directory convention, for instance, makes pass 1 unusually reliable and may warrant raising its confidence threshold.
4. **Extend the benchmark suite.** Add representative repositories in the new language to §41.1 so that regressions are detectable.

WHAT REQUIRES NO CHANGE

Chunking, embedding, indexing, Map-Reduce summarisation, RAG grounding, graph layout, the schema, the API and the frontend all operate on language-normalised structural records and require no modification whatsoever. If adding a language ever requires touching those components, it indicates a language-specific assumption has leaked out of the parsing layer — which is itself the defect to fix.

49. Open Research Questions

Questions this design raises but does not answer. They are recorded because an honest specification distinguishes what was decided from what was merely chosen for now.

49.1 Is 0.60 the right threshold, and should it be fixed at all?

The gate value was set empirically as the point below which retrieved chunks stopped being reliably topically relevant. It is not calibrated against a labelled relevance dataset, and ADR-004 rejected adaptive thresholds for lack of a calibration signal. Building that dataset — question, repository, correct answer, supporting span — would allow the threshold to be optimised against a measurable precision-recall trade rather than set by judgement. This is genuine research work, not tuning.

49.2 How should module clustering quality be measured objectively?

§41 evaluates boundaries by manual review, which does not scale and is not reproducible across reviewers. A quantitative clustering-quality metric — analogous to modularity or silhouette scores in unsupervised learning, but adapted to the asymmetry of directed import graphs — would let the two-pass algorithm's parameters be optimised against an objective signal. Whether standard graph-clustering metrics transfer meaningfully to code dependency graphs is an open question.

49.3 Would code-specialised embeddings materially improve retrieval?

A general-purpose text embedding model is used (§27.1). Code-specialised alternatives plausibly outperform it on this domain, but the comparison cannot be made honestly until the evaluation harness in §41 exists and produces a stable baseline. Sequencing matters here: swapping the model before the harness exists would replace one unmeasured choice with another.

49.4 Can architectural patterns be classified rather than described?

Insights currently emit free-text observations. Aggregated across many analysed repositories, could the intelligence model support classifying a repository into a labelled architectural pattern — layered, hexagonal, event-driven — with a confidence score? This would make insights comparable across repositories rather than merely readable, but it requires a labelled corpus that does not currently exist.

49.5 What is the actual comprehension outcome?

The entire product rests on a premise — that a structured module model reduces comprehension time — which this specification asserts and does not measure. A controlled study comparing time-to-first-correct-contribution with and without RepoLens would test the premise directly. Every metric in §42 measures the system; none measures whether the system achieves its purpose. That gap is worth stating plainly.

A. Glossary

Repository Intelligence

The complete structured model derived from a repository — modules, dependencies, chunks, summaries, insights and learning path. The system's core data product and the single artefact every consumer reads.

Modular monolith

An architecture in which all backend logic runs in one deployable process but is internally partitioned into strictly separated modules with explicit interfaces and no circular dependencies.

Deterministic grounding

The constraint (DG-1) that factual structural claims originate from static analysis, and that generative models may only explain structure already established.

Grounding gate

The hard similarity threshold that aborts answer generation when retrieved evidence is insufficient, implemented as a branch in application code ahead of the model call.

Chunk

A semantically complete unit of source code — a whole function, class, interface or configuration block — bounded at AST nodes and carrying its file path and line span as provenance.

Module candidate

A cluster of files produced by deterministic detection before an LLM assigns it a name, description and responsibility. Membership is fixed before naming begins.

Map-Reduce summarisation

Hierarchical summarisation in which many chunk groups are summarised independently and in parallel, then combined into a single coherent synthesis.

Cache hit / miss

Whether a repository's remote HEAD commit SHA matches the SHA of a stored analysis. A hit serves persisted intelligence; a miss triggers the full pipeline.

System of record

The authoritative durable store. In RepoLens this is PostgreSQL exclusively; every other data structure is derived and disposable.

Derived index

A data structure computable entirely from the system of record, whose loss costs recomputation rather than data. The FAISS index is the sole example.

Failure asymmetry

The property that generative-layer failure degrades the product to its deterministic capabilities rather than failing it entirely, achieved by ordering persistence before generation.

Cosine similarity

A similarity measure between vectors based on the angle between them. On L2-normalised vectors it is identical to inner product, which is why `IndexFlatIP` is used.

B. Acronyms

Acronym	Expansion	Context in this document
ADR	Architecture Decision Record	§14, Appendix C
AST	Abstract Syntax Tree	Tree-sitter output; the basis of all structural extraction (§24)
CST	Concrete Syntax Tree	Lossless parse representation preceding AST simplification
DG-1	Deterministic Grounding constraint 1	§5.1, the system's organising constraint
EDD	Engineering Design Document	One of this document's roles
FR / NFR	Functional / Non-Functional Requirement	§9, §10
GFM	GitHub-Flavored Markdown	Export format (§38)
JWT	JSON Web Token	Session mechanism (§35.1)
MTEB	Massive Text Embedding Benchmark	Basis for embedding model selection (§27.1)
ORM	Object-Relational Mapper	SQLAlchemy (§32)
PRD	Product Requirements Document	One of this document's roles
RAG	Retrieval-Augmented Generation	§29
SHA	Secure Hash Algorithm — here, a Git commit identifier	Cache key (§37)
SRS	Software Requirements Specification	One of this document's roles
TLS	Transport Layer Security	All inter-provider communication (§17)

C. Complete ADR Index

TABLE C.1 — ALL ARCHITECTURE DECISIONS, CONSOLIDATED

ID	Decision	Rejected alternative	Status	Detail
ADR-001	Modular monolith	Microservices; serverless functions	Accepted	§14
ADR-002	PostgreSQL as record, FAISS derived	Managed vector DB; disk-persisted index	Accepted	§14
ADR-003	Deterministic extraction before generation	Direct LLM structural inference	Accepted	§14
ADR-004	Similarity gate ahead of the model	Prompt-instructed self-regulation; adaptive threshold	Accepted	§14
ADR-005	Hosted Gemini 2.5 Flash	Self-hosted open-weight model	Accepted	§14
ADR-006	Tree-sitter over native parsers	Four native language front-ends	Accepted	§14
ADR-007	Polled status column	Server-sent events; websockets	Accepted	§14
ADR-008	Normalised inner product retrieval	Euclidean distance on raw vectors	Accepted	§14
ADR-009	Stateless JWT sessions	Server-side session store	Accepted	No revocation requirement exists; avoids a stateful dependency (§35.1)
ADR-010	Stateless multi-turn Q&A	Persisted conversation table	Accepted	Client sends last three turns; avoids schema, retention policy and a privacy surface (§29.4)
ADR-011	Exhaustive IndexFlatIP	Approximate index — HNSW, IVF	Accepted	Exact retrieval with zero tuning inside the current envelope; revisit per §46
ADR-012	Path-prefix API versioning	Header or content negotiation	Accepted	Legible in logs and routable; alternatives solve a multi-consumer problem we do not have (§47)
ADR-013	Server-side graph layout	Client-side Dagre execution	Accepted	Layout computed once per analysis rather than once per page load; identical across clients (§30)
ADR-014	No third caching layer	Redis HTTP response cache	Accepted	Both dominant cost centres already addressed; a third layer adds invalidation bugs for no measured gain (§37)

D. API Quick Reference

AUTHENTICATION

GET /api/v1/auth/github → 302 to GitHub, read-only scopes
 GET /api/v1/auth/callback?code=... → { jwt, user }

REPOSITORIES

GET /api/v1/repositories → [{ owner, name, language, size, ... }]

ANALYSIS

POST /api/v1/analysis/trigger { owner, name }
 → 200 { cached:true, repository_id }
 → 202 { cached:false, analysis_id }
 → 413 { error:"repository_too_large" }
 GET /api/v1/analysis/{id}/status → { stage, progress, updated_at }

INTELLIGENCE

GET /api/v1/intelligence/{repo_id} → full RepositoryIntelligence model
 GET /api/v1/map/{repo_id} → { nodes:[{id,label,x,y,...}], edges:[...] }

SEARCH AND ANSWERING

POST /api/v1/search/semantic { repository_id, query, k? }
 → { results:[{chunk,path,lines,score}] }
 POST /api/v1/ask { repository_id, query, history[] }
 → { grounded, answer, citations[] }

REPORTS

POST /api/v1/report/export { repository_id, format:"pdf"|"markdown" }
 → binary artefact

CONVENTIONS

- Bearer JWT on every route except /auth/*
- Errors: { error:<machine_code>, message:<human_safe> } – never a stack trace
- Breaking changes require /api/v2; v1 is never modified in place

E. Database Schema (DDL)

```

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  github_id   BIGINT UNIQUE NOT NULL,
  username    VARCHAR(255) NOT NULL,
  avatar_url  TEXT,
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_users_github_id ON users(github_id);

CREATE TYPE analysis_status AS ENUM
  ('QUEUED', 'PROCESSING', 'COMPLETED', 'FAILED');

CREATE TABLE repositories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  owner       VARCHAR(255) NOT NULL,
  primary_language VARCHAR(50),
  commit_sha  VARCHAR(64),
  status      analysis_status NOT NULL DEFAULT 'QUEUED',
  analyzed_at  TIMESTAMP,
  UNIQUE (user_id, owner, name)
);
CREATE INDEX idx_repo_sha    ON repositories(commit_sha);
CREATE INDEX idx_repo_status ON repositories(status);

CREATE TABLE repository_intelligence (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id  UUID NOT NULL UNIQUE
    REFERENCES repositories(id) ON DELETE CASCADE,
  summary      TEXT,
  tech_stack_json  JSONB,
  architecture_summary TEXT,
  learning_path_json  JSONB,
  insights_json  JSONB
);

CREATE TABLE modules (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id  UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  description  TEXT,
  category     VARCHAR(100),
  file_paths   JSONB NOT NULL,
  relationships_json  JSONB
);
CREATE INDEX idx_modules_repo ON modules(repository_id);

CREATE TABLE repository_chunks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id  UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  file_path     VARCHAR(1024) NOT NULL,
  chunk_content TEXT NOT NULL,
  start_line    INT NOT NULL,
  end_line      INT NOT NULL,
  embedding_vector  FLOAT4[] NOT NULL      -- 384 L2-normalised floats
);
-- Performance-critical: backs FAISS index reconstruction (§16.2)

```

```
CREATE INDEX idx_chunks_repo ON repository_chunks(repository_id);

CREATE TYPE report_format AS ENUM ('PDF','MARKDOWN');

CREATE TABLE reports (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  report_type  report_format NOT NULL,
  file_path    TEXT NOT NULL,
  generated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

CASCADE SEMANTICS

Every child table cascades on repository deletion. Deleting a repository must remove its modules, chunks, vectors, intelligence and reports atomically — orphaned chunks would be invisible to the application while still consuming storage and, worse, could be reconstructed into a FAISS index and returned as results for a repository that no longer exists.

F. References

- [1] Brunsfeld, M. et al. (2023). Tree-sitter: An Incremental Parsing System for Programming Tools. Official documentation and grammar specification. <https://tree-sitter.github.io/tree-sitter/>
- [2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33, 9459–9474.
- [3] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. (FAISS reference implementation.)
- [4] Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged Resources to Advance General Chinese Embedding. Beijing Academy of Artificial Intelligence. [arXiv:2309.07597](https://arxiv.org/abs/2309.07597). (BAAI/bge-small-en-v1.5 specification.)
- [5] Sugiyama, K., Tagawa, S., & Toda, M. (1981). Methods for Visual Understanding of Hierarchical System Structures. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-11(2), 109–125. (Foundation of the Dagre layered layout algorithm.)
- [6] Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. *Proceedings of EACL 2023*. (Basis for the embedding model comparison in §27.1.)
- [7] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*. (Foundation of the sentence-transformers inference path.)
- [8] Ramírez, S. (2018–). FastAPI: High performance, easy to learn, fast to code, ready for production. <https://fastapi.tiangolo.com/>
- [9] webkid GmbH (2024). React Flow: A customizable library for building node-based user interfaces. <https://reactflow.dev/>
- [10] Kozea (2024). WeasyPrint: A visual rendering engine for HTML and CSS to PDF. <https://weasyprint.org/>
- [11] Fowler, M. (2015). MonolithFirst and Microservice Trade-Offs. martinfowler.com. (Referenced in the reasoning behind ADR-001.)
- [12] Nygard, M. (2011). Documenting Architecture Decisions. (Origin of the ADR format used in §14 and Appendix C.)

END OF SPECIFICATION · RepoLens v2.0 · Engineering Design Document, Product Requirements Document, Software Requirements Specification